



SI100B: Introduction to Information Science and Technology



Lecture 8

Announcement

- ▶ Homework 2
 - ▶ Instructions & problems
 - ▶ BB → 作业 → “Programming Assignment 2”
 - ▶ Submission website: OJ
 - ▶ Due: 9pm, Oct. 23





ALIASING, CLONING

OPERATION ON LISTS: `remove`

- ▶ Delete element at a **specific index** with `del(L[index])`
- ▶ Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- ▶ Remove a **specific element** with `L.remove(element)`
 - ▶ Looks for the element and removes it (mutating the list)
 - ▶ If element occurs multiple times, removes first occurrence
 - ▶ If element not in list, gives an error

`L = [2, 1, 3, 6, 3, 7, 0]` # do below in order

`L.remove(2)` → mutates `L = [1, 3, 6, 3, 7, 0]`

`L.remove(3)` → mutates `L = [1, 6, 3, 7, 0]`

`del(L[1])` → mutates `L = [1, 3, 7, 0]`

`a = L.pop()` → returns 0 and mutates `L = [1, 3, 7]`



OPERATION ON LISTS: `remove`

- ▶ Delete element at a **specific index** with `del(L[index])`
- ▶ Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- ▶ Remove a **specific element** with `L.remove(element)`
 - ▶ Looks for the element and removes it (mutating the list)
 - ▶ If element occurs multiple times, removes first occurrence
 - ▶ If element not in list, gives an error

`L = [2, 1, 3, 6, 3, 7, 0]` # do below in order

`L.remove(2)` → mutates `L = [1, 3, 6, 3, 7, 0]`

`L.remove(3)` → mutates `L = [1, 6, 3, 7, 0]`

`del(L[1])` → mutates `L = [1, 3, 7, 0]`

`a = L.pop()` → returns 0 and mutates `L = [1, 3, 7]`



OPERATION ON LISTS: `remove`

- ▶ Delete element at a **specific index** with `del(L[index])`
- ▶ Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- ▶ Remove a **specific element** with `L.remove(element)`
 - ▶ Looks for the element and removes it (mutating the list)
 - ▶ If element occurs multiple times, removes first occurrence
 - ▶ If element not in list, gives an error

`L = [2, 1, 3, 6, 3, 7, 0]` # do below in order

`L.remove(2)` → mutates `L = [1, 3, 6, 3, 7, 0]`

`L.remove(3)` → mutates `L = [1, 6, 3, 7, 0]`

`del(L[1])` → mutates `L = [1, 3, 7, 0]`

`a = L.pop()` → returns 0 and mutates `L = [1, 3, 7]`



OPERATION ON LISTS: `remove`

- ▶ Delete element at a **specific index** with `del (L[index])`
- ▶ Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- ▶ Remove a **specific element** with `L.remove(element)`
 - ▶ Looks for the element and removes it (mutating the list)
 - ▶ If element occurs multiple times, removes first occurrence
 - ▶ If element not in list, gives an error

`L = [2, 1, 3, 6, 3, 7, 0]` # do below in order

`L.remove(2)` → mutates `L = [1, 3, 6, 3, 7, 0]`

`L.remove(3)` → mutates `L = [1, 6, 3, 7, 0]`

`del (L[1])` → mutates `L = [1, 3, 7, 0]`

`a = L.pop()` → returns 0 and mutates `L = [1, 3, 7]`



OPERATION ON LISTS: `remove`

- ▶ Delete element at a **specific index** with `del(L[index])`
- ▶ Remove element at **end of list** with `L.pop()`, returns the removed element (can also call with specific index: `L.pop(3)`)
- ▶ Remove a **specific element** with `L.remove(element)`
 - ▶ Looks for the element and removes it (mutating the list)
 - ▶ If element occurs multiple times, removes first occurrence
 - ▶ If element not in list, gives an error

all these
operations
mutate
the list

`L = [2, 1, 3, 6, 3, 7, 0]` # do below in order

`L.remove(2)` → mutates `L = [1, 3, 6, 3, 7, 0]`

`L.remove(3)` → mutates `L = [1, 6, 3, 7, 0]`

`del(L[1])` → mutates `L = [1, 3, 7, 0]`

`a = L.pop()` → returns 0 and mutates `L = [1, 3, 7]`



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

- ▶ Rewrite the code to remove `e` as long as we still had it in the list
- ▶ It works well!

```
def remove_all(L, e):  
    """  
    L is a list  
    Mutates L to remove all elements in L that are equal to e  
    Returns None.  
    """  
    while e in L:  
        L.remove(e)
```



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

► What if the code was this:

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

L = [1,2,2,2]
remove_all(L, 2)
print(L)      # should print [1]
```



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

► What if the code was this:

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)
```

```
L = [1,2,2,2]
remove_all(L, 2)
print(L) # should print [1]
```

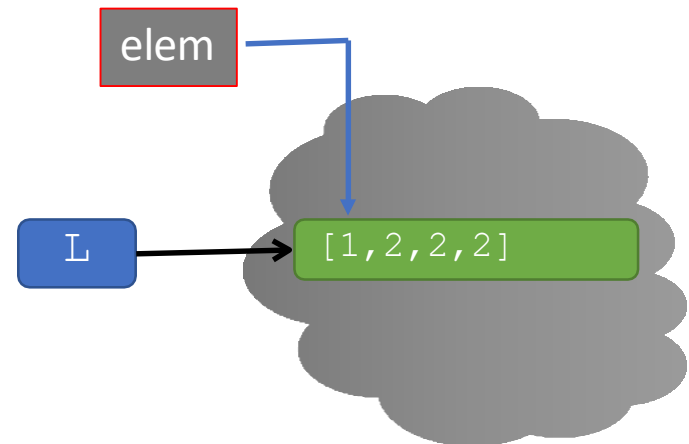
Actually prints [1,2]



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

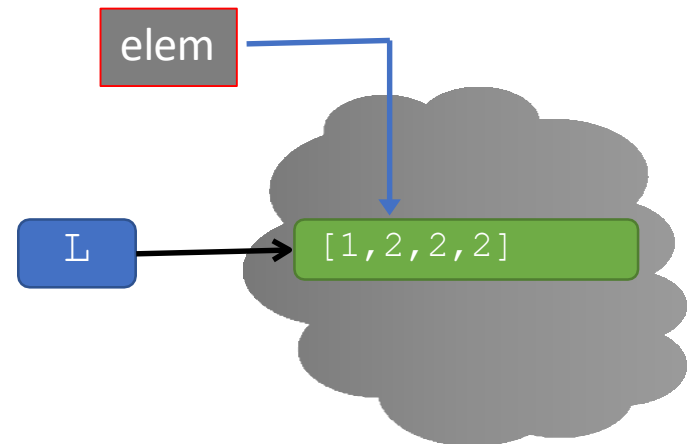
L = [1,2,2,2]
remove_all(L, 2)
print(L)      # should print [1]
```



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

L = [1,2,2,2]
remove_all(L, 2)
print(L)      # should print [1]
```

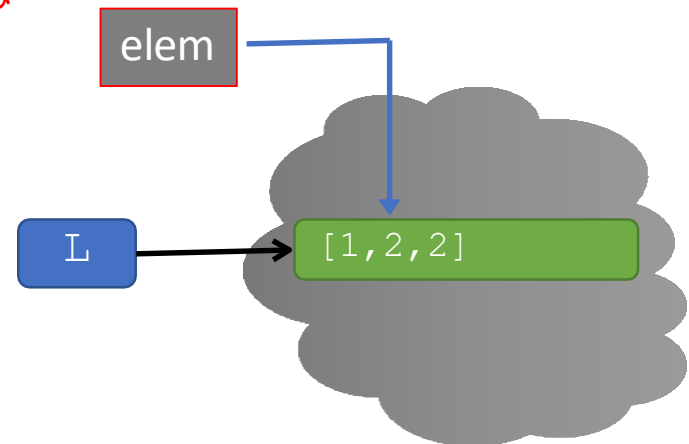


EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

```
def remove_all(L, e): """
    L is a list
    Mutates L to remove all elements in L that are equal to e
    Returns None.
    """
    for elem in L:
        if elem == e:
            L.remove(e)

L = [1,2,2,2]
remove_all(L, 2)
print(L)      # should print [1]
```

*Removes the 2, but
doesn't shift the
pointer back!*



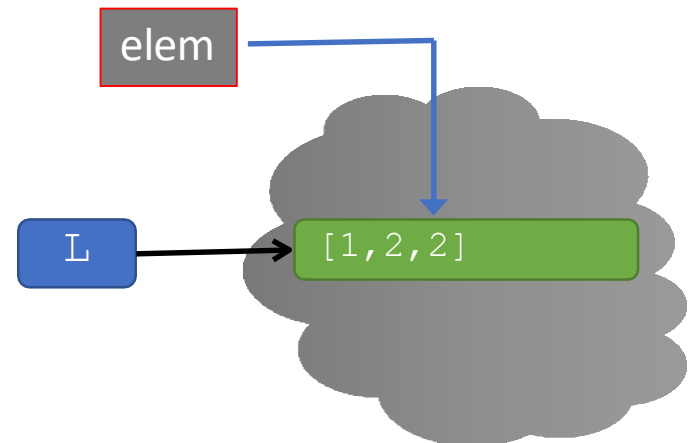
EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

```
def remove_all(L, e): """  
    L is a list  
    Mutates L to remove all elements in L that are equal to e  
    Returns None.  
    """
```

```
    for elem in L:  
        if elem == e:  
            L.remove(e)
```

*elem moves
forward in
the sequence*

```
L = [1,2,2,2]  
remove_all(L, 2)  
print(L)      # should print [1]
```



EXERCISE WITH REMOVE INSTEAD OF COPY AND CLEAR

- It's not correct! We **removed items as we iterated over the list!**

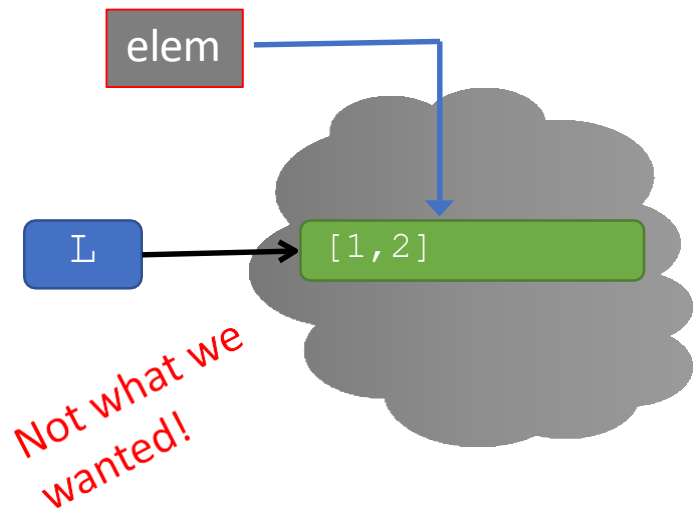
```
def remove_all(L, e): """  
    L is a list  
    Mutates L to remove all elements in L that are equal to e  
    Returns None.  
    """
```

```
    for elem in L:
```

```
        if elem == e:  
            L.remove(e)
```

*Remove the
2, and done*

```
L = [1,2,2,2]  
remove_all(L, 2)  
print(L)      # should print [1]
```



TRICKY EXAMPLE 4

- ▶ Want to mutate L1 to remove any elements that are also in L2

```
def remove_dups(L1, L2):  
    for e in L1:
```

```
         if e in L2:
```

```
        L1.remove(e)
```

```
L1 = [10, 20, 30, 40]
```

```
L2 = [10, 20, 50, 60]
```

```
remove_dups(L1, L2)
```

*Want a function that returns a list,
where every element also in a
second list is removed.*

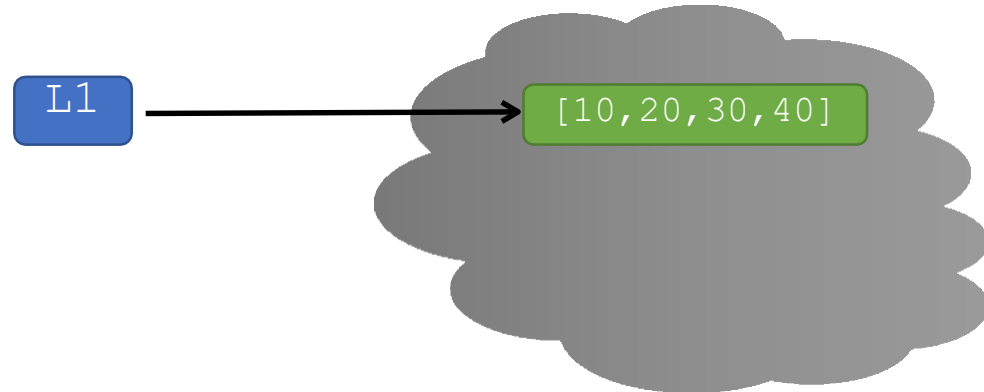
- ▶ L1 is [20, 30, 40] **not** [30, 40]
- ▶ Why?



MUTATION AND ITERATION WITHOUT CLONE

```
def remove_dups(L1, L2):  
    for e in L1:  
        if e in L2:  
            L1.remove(e)
```

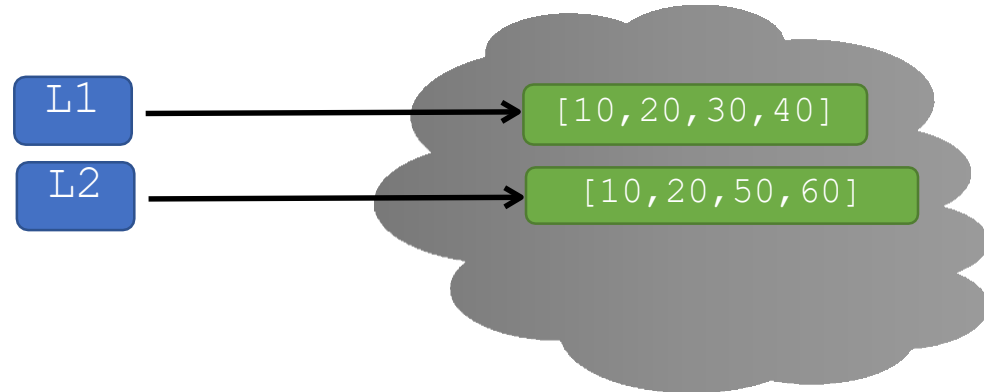
```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



MUTATION AND ITERATION WITHOUT CLONE

```
def remove_dups(L1, L2):  
    for e in L1:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```

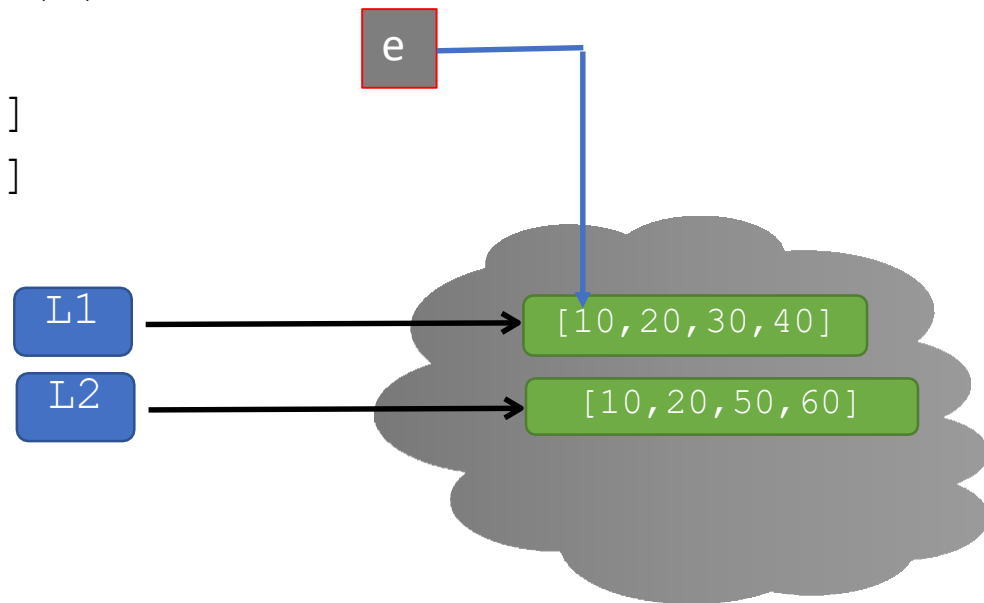


MUTATION AND ITERATION WITHOUT CLONE

```
def remove_dups(L1, L2):  
    for e in L1:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```

- Python uses an internal counter, tracking index in the loop over L1

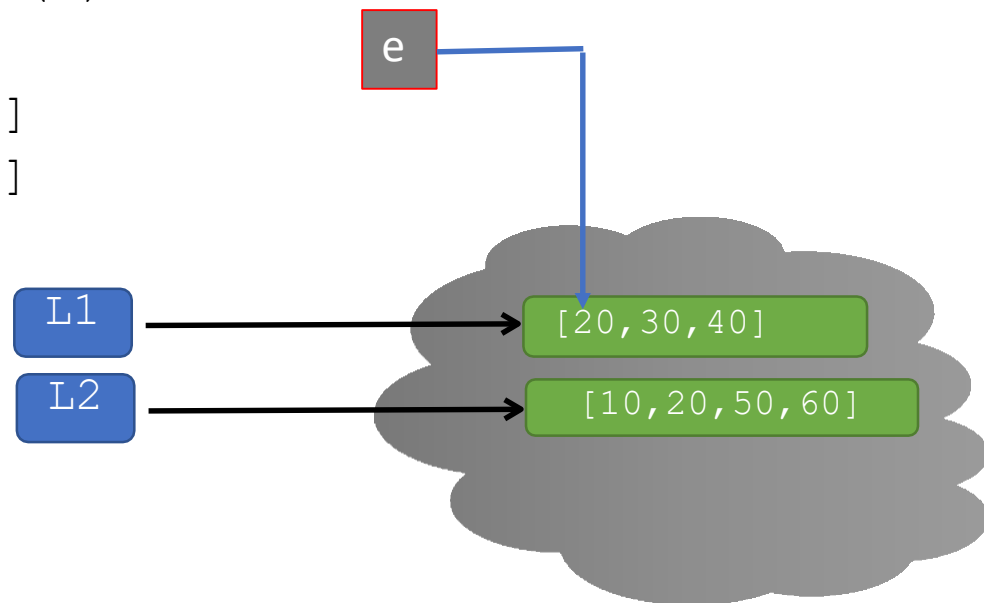


MUTATION AND ITERATION WITHOUT CLONE

```
def remove_dups(L1, L2):  
    for e in L1:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```

- Python uses an internal counter, tracking index in the loop over L1
- Mutating changes the list but Python doesn't update the counter

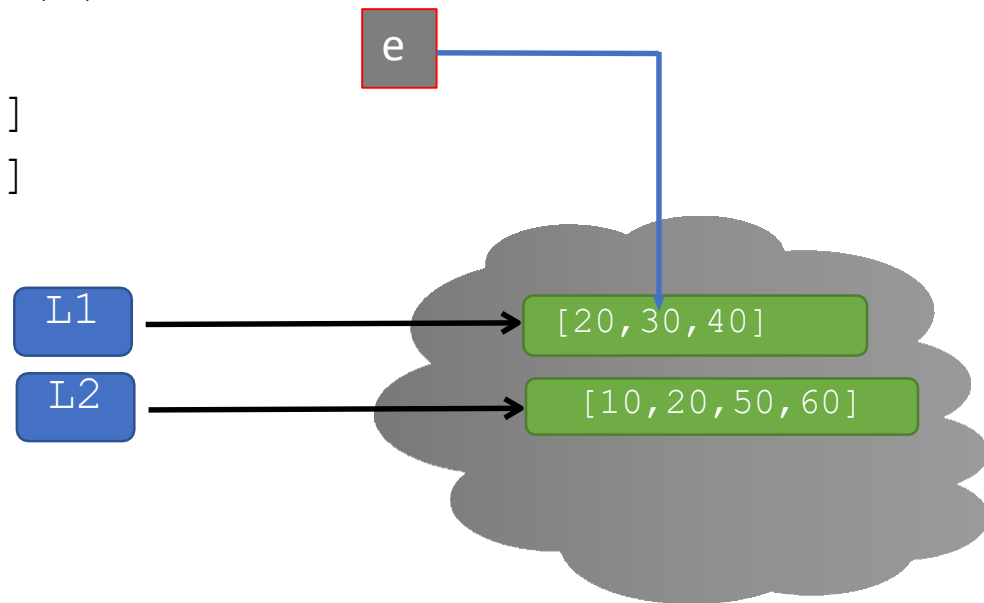


MUTATION AND ITERATION WITHOUT CLONE

```
def remove_dups(L1, L2):  
    for e in L1:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```

- Python uses an internal counter, tracking index in the loop over L1
- Mutating changes the list but Python doesn't update the counter

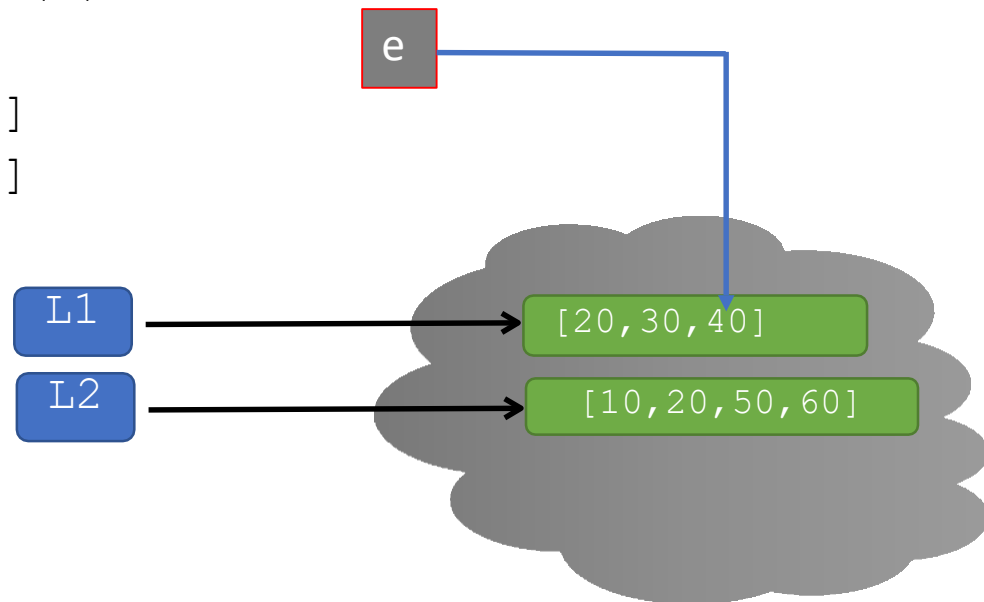


MUTATION AND ITERATION WITHOUT CLONE

```
def remove_dups(L1, L2):  
    for e in L1:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```

- Python uses an internal counter, tracking index in the loop over L1
- Mutating changes the list but Python doesn't update the counter



MUTATION AND ITERATION WITH CLONE

```
L1_copy = L1[:]
```

▶ Make a **clone** with [:]



```
def remove_dups(L1, L2):  
    for e in L1:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



```
def remove_dups(L1, L2):  
    L1_copy = L1[:]   
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

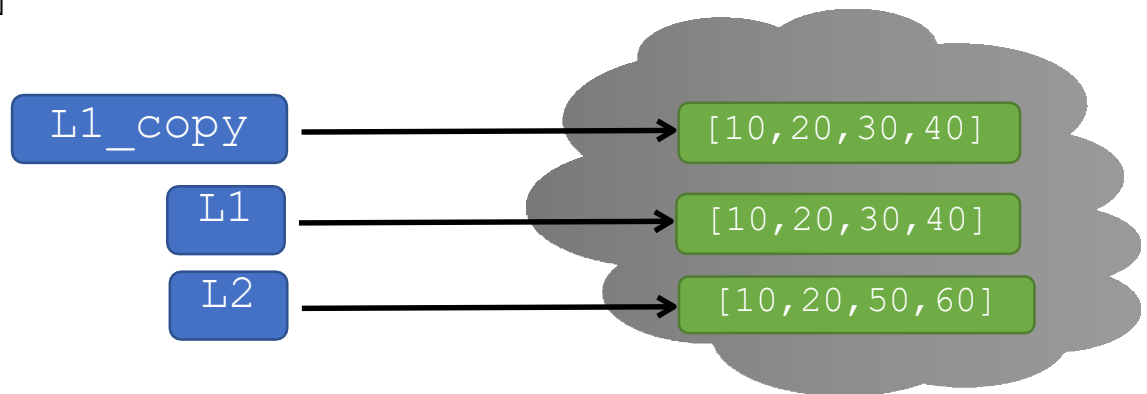
▶ New version works!

- ▶ Iterate over a copy
- ▶ Mutate original list, not the copy
- ▶ Indexing is now consistent



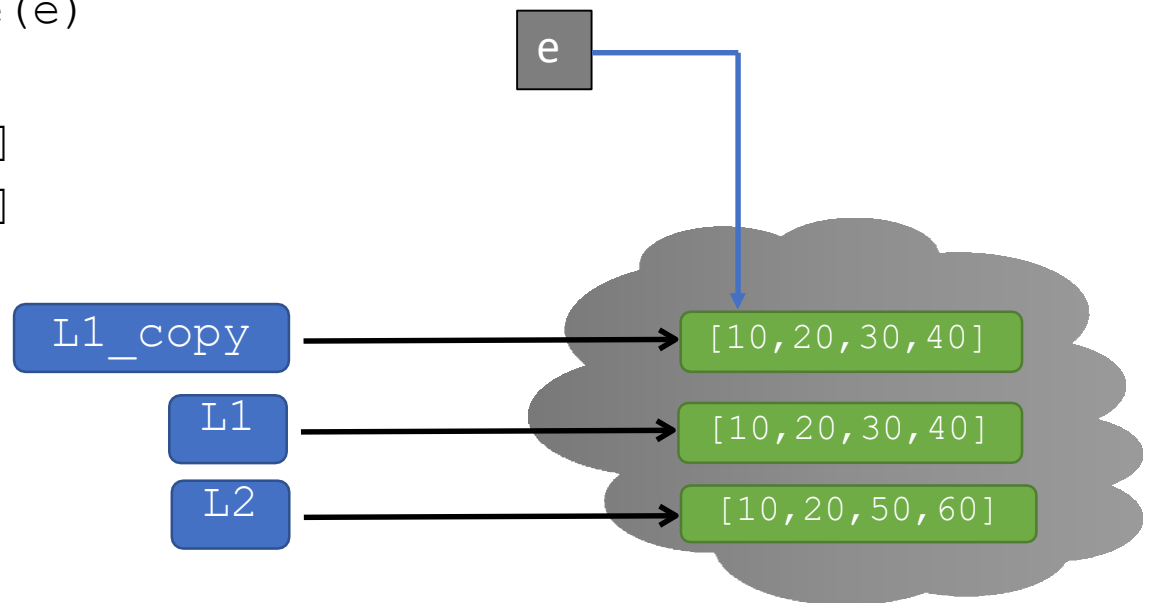
```
def remove_dups(L1, L2):  
    L1_copy = L1[:]  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



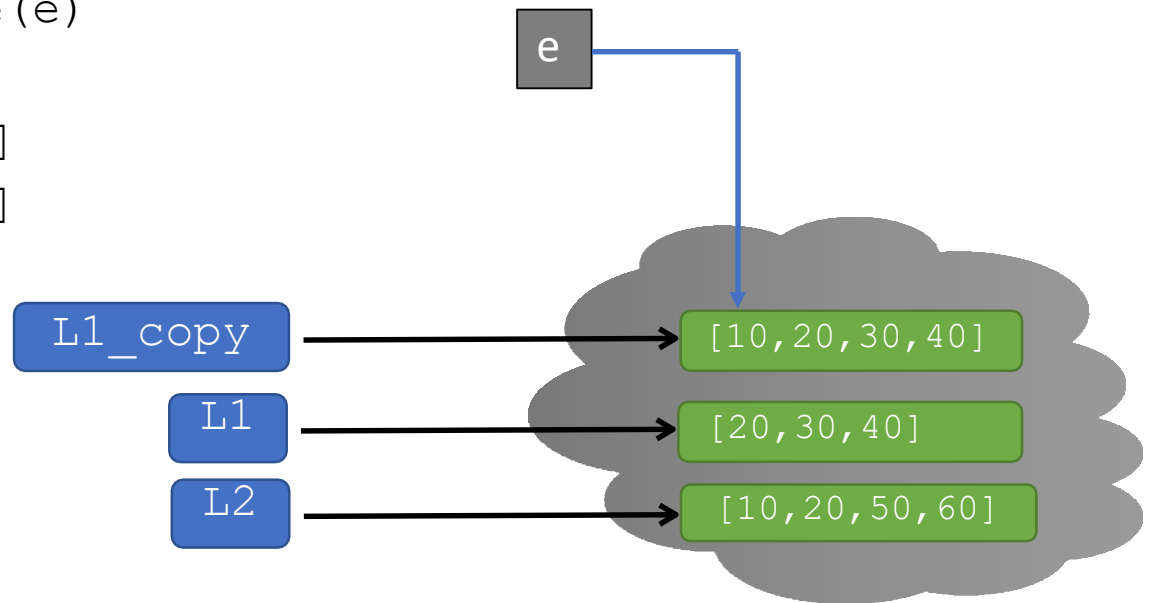
```
def remove_dups(L1, L2):  
    L1_copy = L1[:]  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



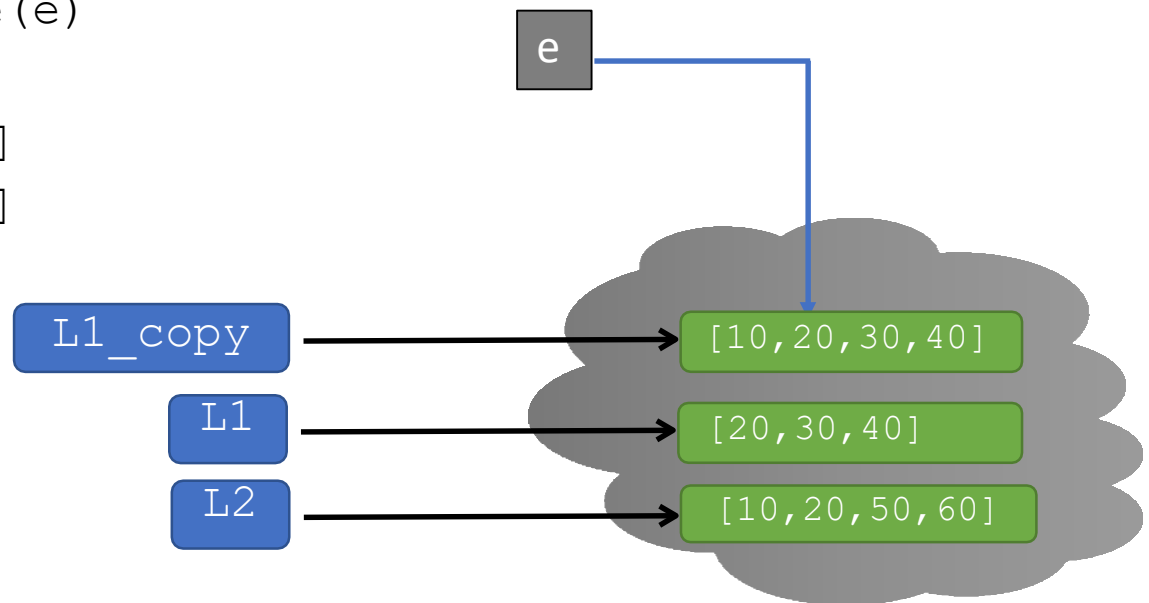
```
def remove_dups(L1, L2):  
    L1_copy = L1[:]  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



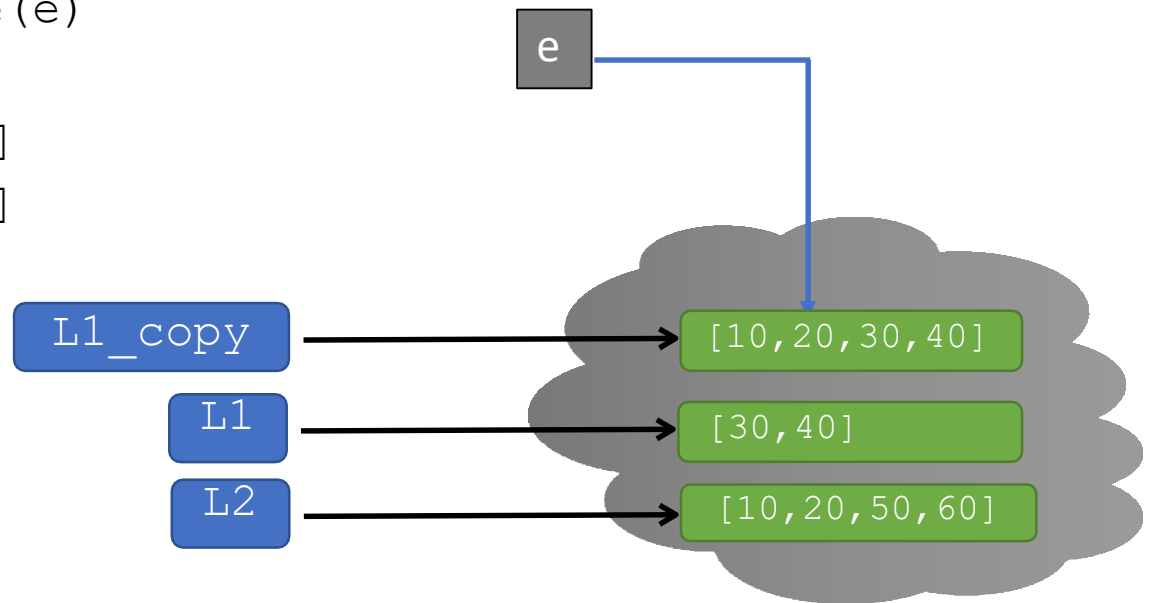
```
def remove_dups(L1, L2):  
    L1_copy = L1[:]  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



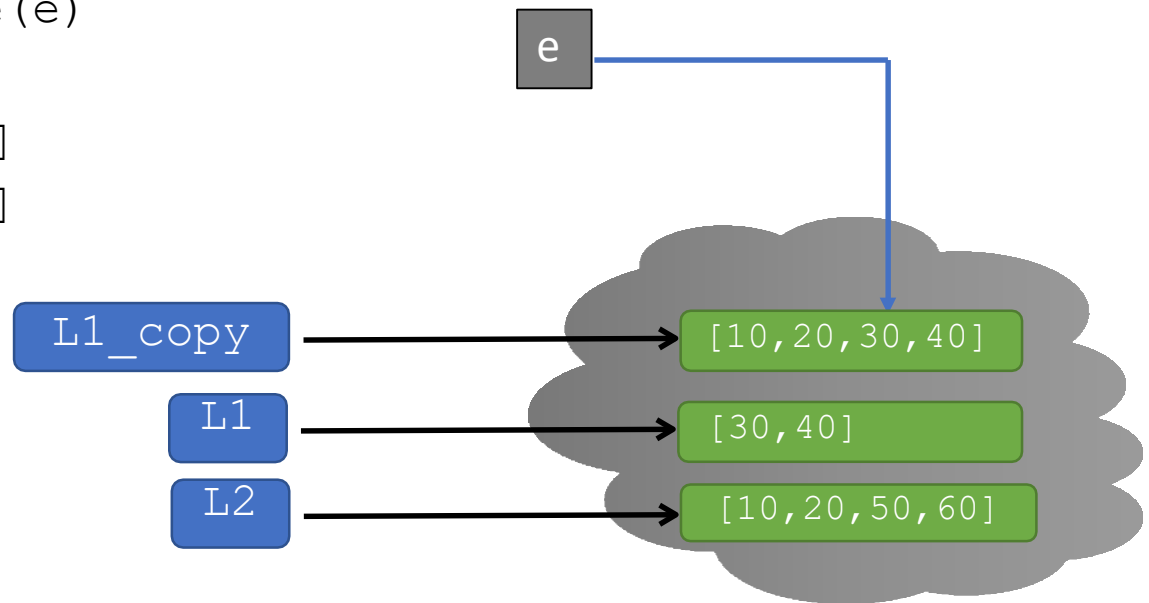
```
def remove_dups(L1, L2):  
    L1_copy = L1[:]  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



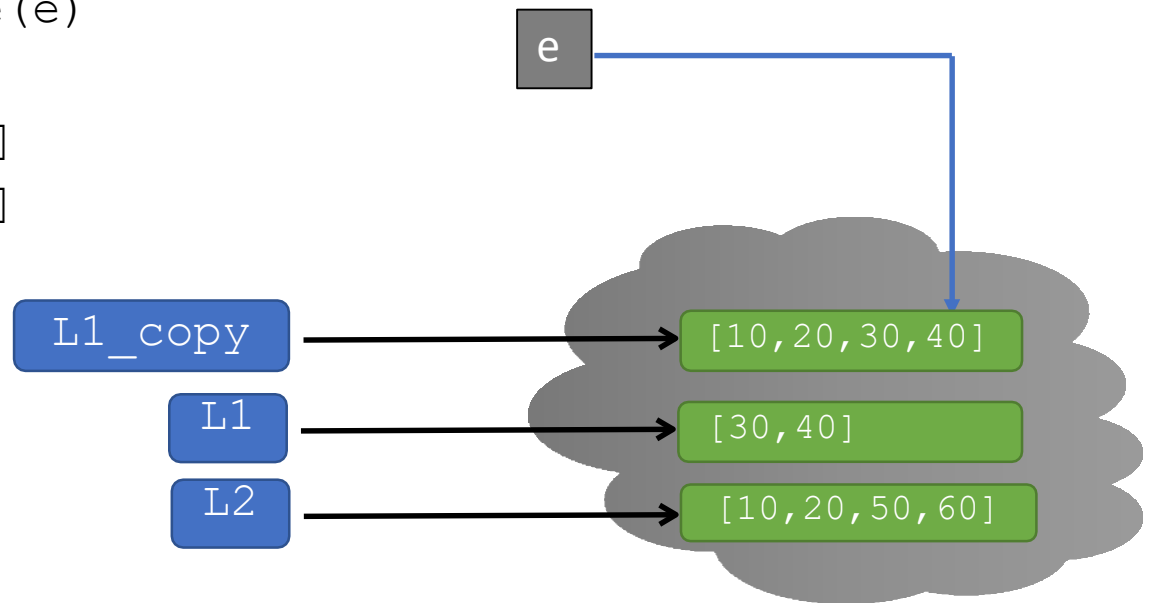
```
def remove_dups(L1, L2):  
    L1_copy = L1[:]  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



```
def remove_dups(L1, L2):  
    L1_copy = L1[:]  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



MUTATION AND ITERATION WITH ALIAS

```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```



The same object as L1

```
def remove_dups(L1, L2):  
    L1_copy = L1[:]  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```



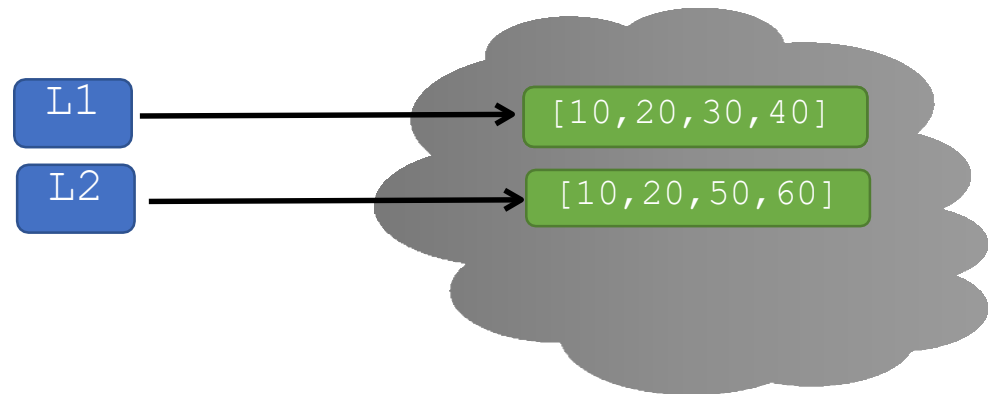
```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```

Note that `L1_copy = L1`
does NOT clone

- ▶ Assignment (= sign) on mutable obj creates an **alias**, not a clone
 - ▶ Same list object referenced by another name
 - ▶ It's like iterating over L itself, it doesn't work!

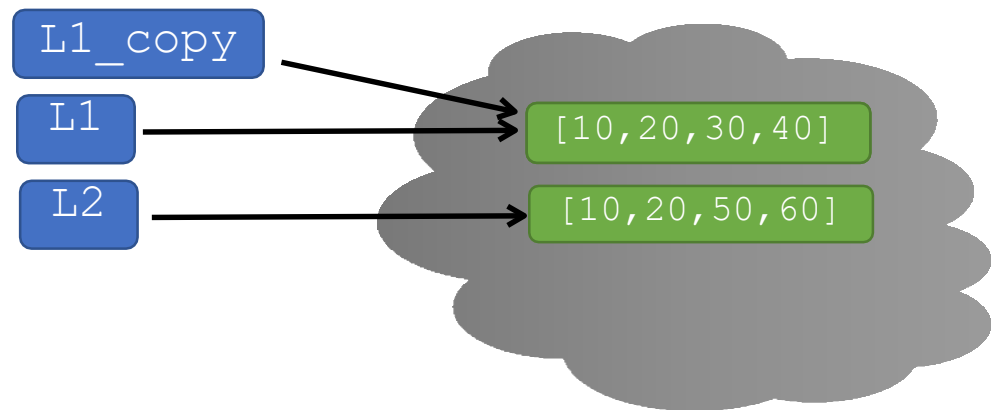
```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



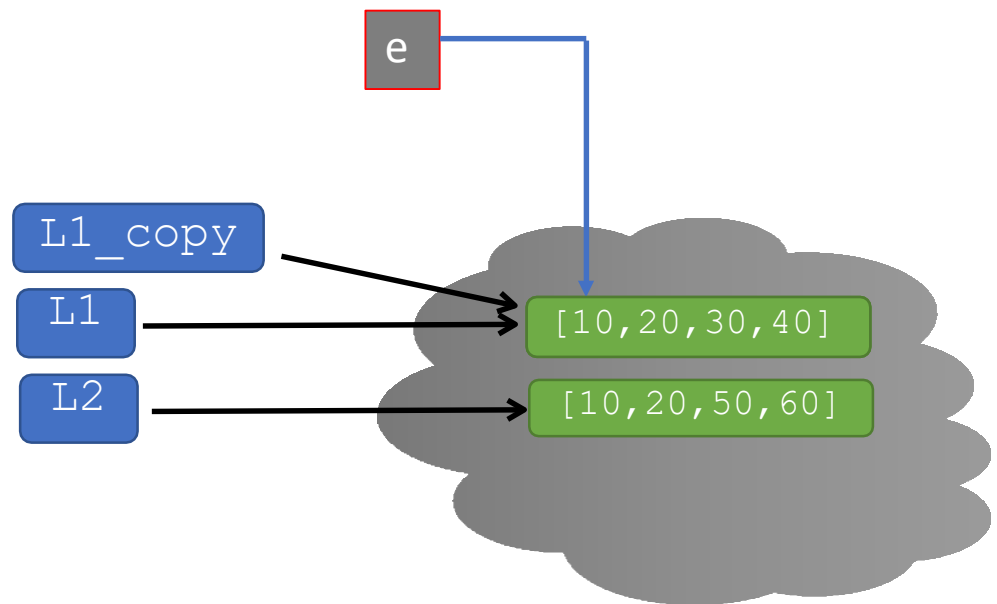
```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



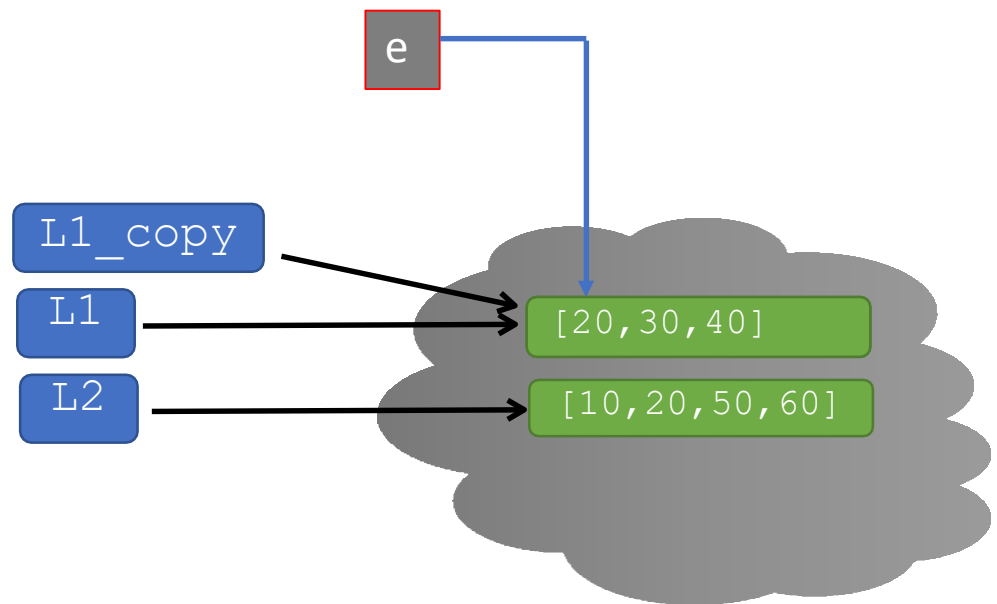
```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



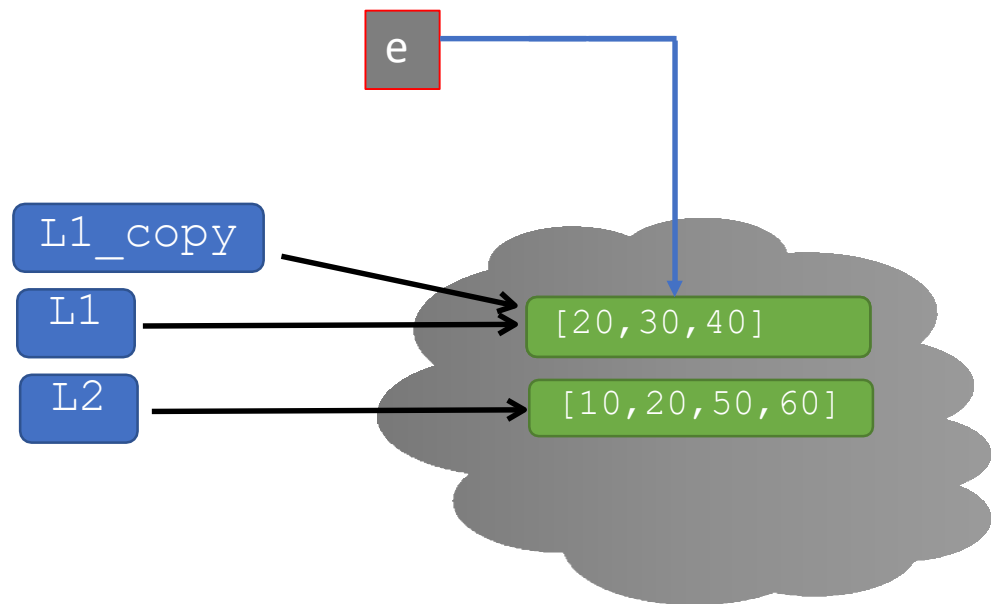
```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



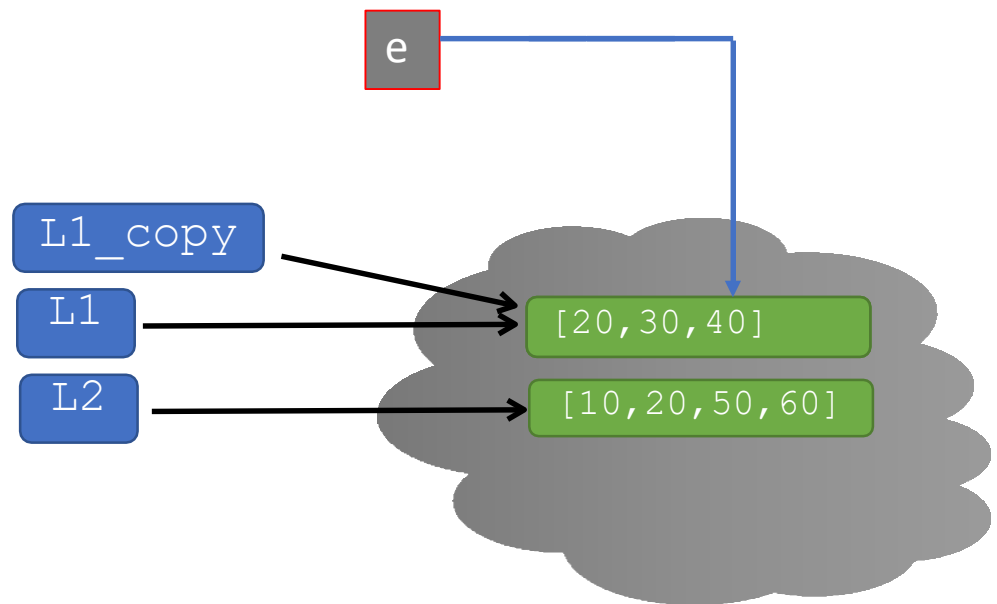
```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
L1 = [10, 20, 30, 40]  
L2 = [10, 20, 50, 60]  
remove_dups(L1, L2)
```



BIG IDEA

When you pass a list as a parameter to a function, you are making an alias.

The **actual parameter** (from the function **call**) is an **alias** for the **formal parameter** (from the function **definition**).



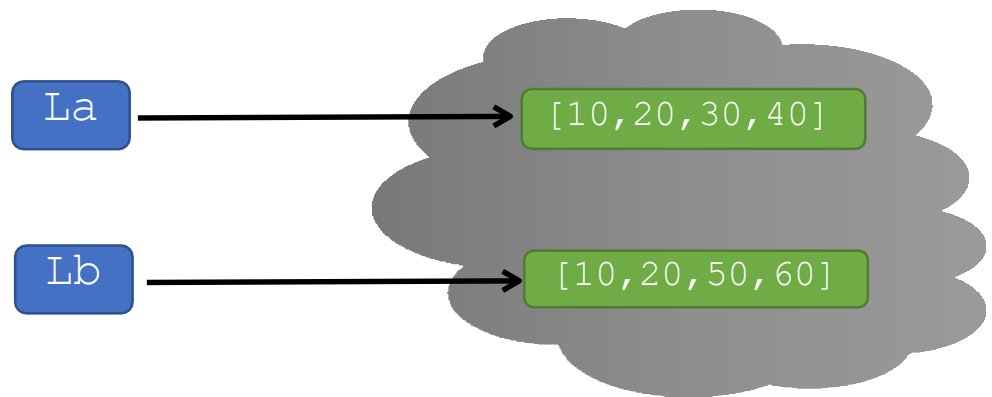
```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
La = [10, 20, 30, 40]  
Lb = [10, 20, 50, 60]  
remove_dups(La, Lb)  
print(La)
```



```
def remove_dups(L1, L2):  
    L1_copy = L1  
    for e in L1_copy:  
        if e in L2:  
            L1.remove(e)
```

```
La = [10, 20, 30, 40]  
Lb = [10, 20, 50, 60]  
remove_dups(La, Lb)  
print(La)
```



```
def remove_dups(L1, L2):
```

Alias for La

```
    L1_copy = L1
```

```
    for e in L1_copy:
```

```
        if e in L2:
```

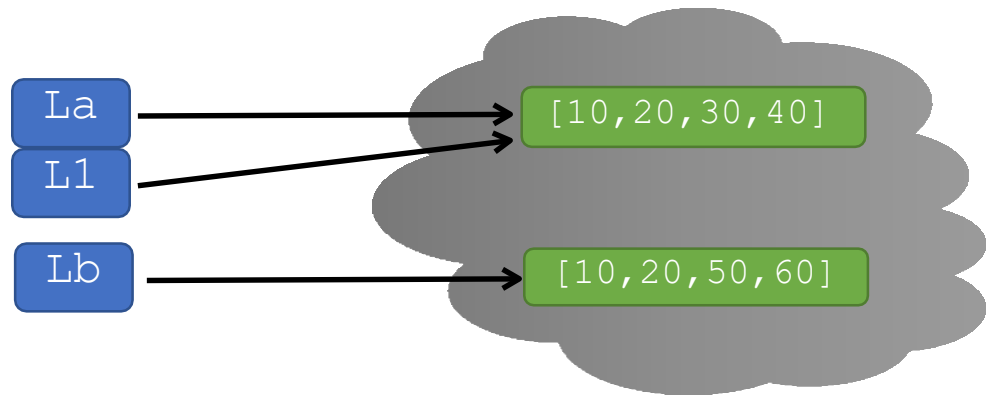
```
            L1.remove(e)
```

```
La = [10, 20, 30, 40]
```

```
Lb = [10, 20, 50, 60]
```

```
remove_dups(La, Lb)
```

```
print(La)
```



```
def remove_dups(L1, L2):
```

Alias for La

Alias for Lb

```
    L1_copy = L1
```

```
    for e in L1_copy:
```

```
        if e in L2:
```

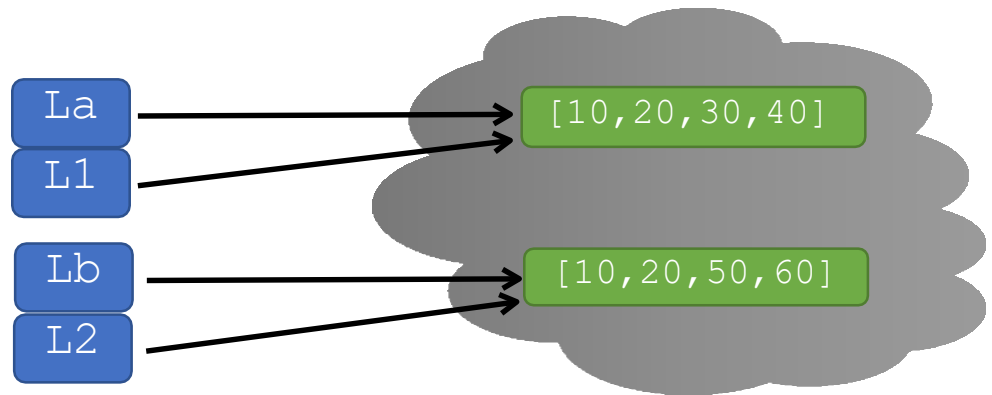
```
            L1.remove(e)
```

```
La = [10, 20, 30, 40]
```

```
Lb = [10, 20, 50, 60]
```

```
remove_dups(La, Lb)
```

```
print(La)
```



```
def remove_dups(L1, L2):
```

Alias for La
Alias for Lb

```
    L1_copy = L1
```

```
    for e in L1_copy:
```

```
        if e in L2:
```

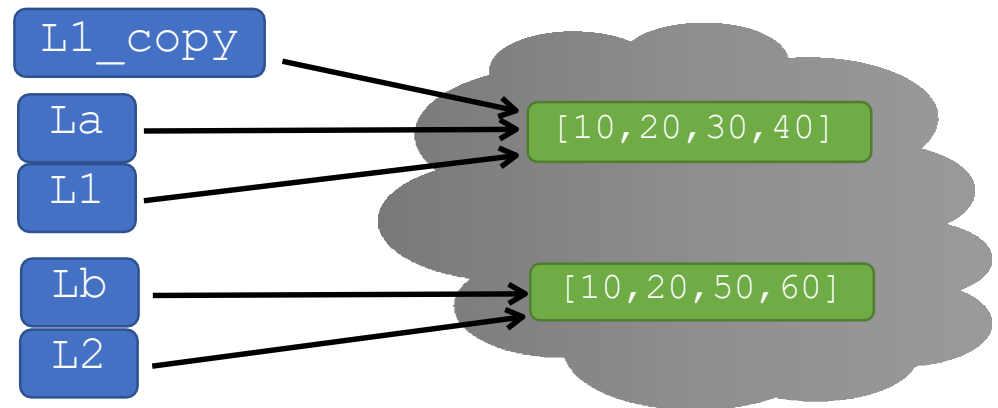
```
            L1.remove(e)
```

```
La = [10, 20, 30, 40]
```

```
Lb = [10, 20, 50, 60]
```

```
remove_dups(La, Lb)
```

```
print(La)
```



```
def remove_dups(L1, L2):
```

Alias for La
Alias for Lb

```
    L1_copy = L1
```

```
    for e in L1_copy:
```

```
        if e in L2:
```

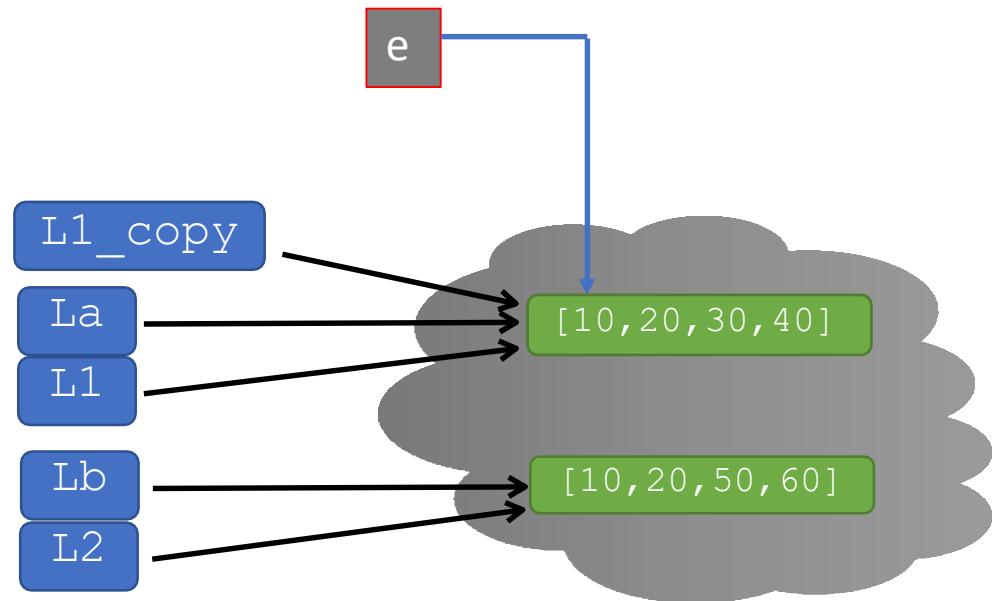
```
            L1.remove(e)
```

```
La = [10, 20, 30, 40]
```

```
Lb = [10, 20, 50, 60]
```

```
remove_dups(La, Lb)
```

```
print(La)
```



```
def remove_dups(L1, L2):
```

Alias for La
Alias for Lb

```
    L1_copy = L1
```

```
    for e in L1_copy:
```

```
        if e in L2:
```

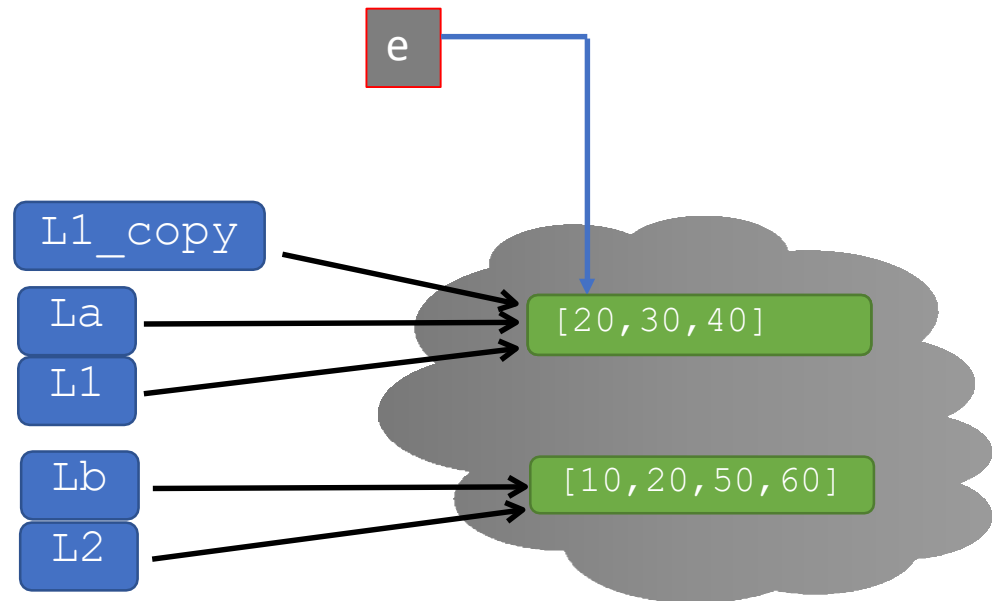
```
            L1.remove(e)
```

```
La = [10, 20, 30, 40]
```

```
Lb = [10, 20, 50, 60]
```

```
remove_dups(La, Lb)
```

```
print(La)
```



```
def remove_dups(L1, L2):
```

Alias for La
Alias for Lb

```
    L1_copy = L1
```

```
    for e in L1_copy:
```

```
        if e in L2:
```

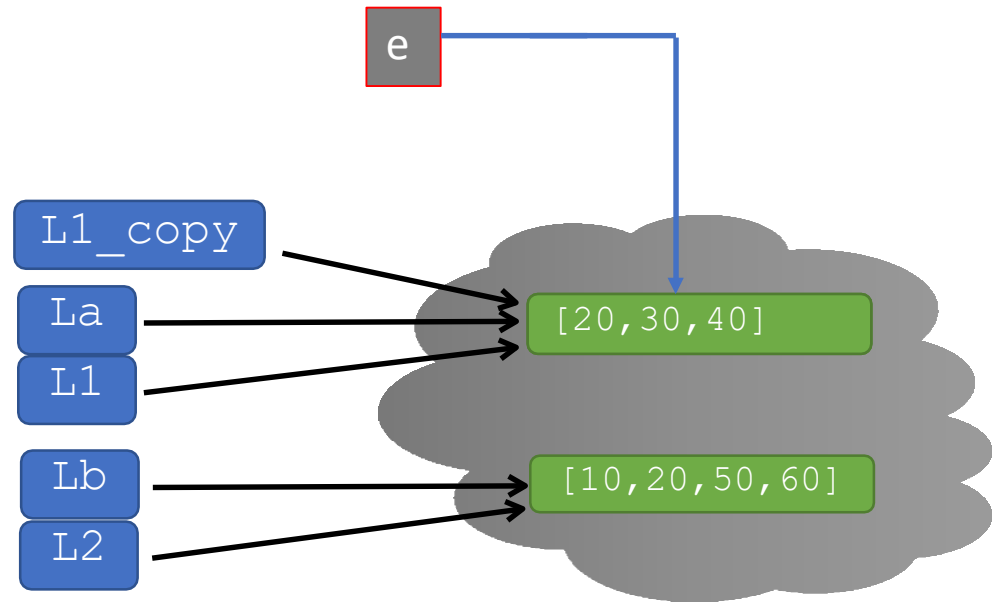
```
            L1.remove(e)
```

```
La = [10, 20, 30, 40]
```

```
Lb = [10, 20, 50, 60]
```

```
remove_dups(La, Lb)
```

```
print(La)
```



```
def remove_dups(L1, L2):
```

Alias for La
Alias for Lb

```
    L1_copy = L1
```

```
    for e in L1_copy:
```

```
        if e in L2:
```

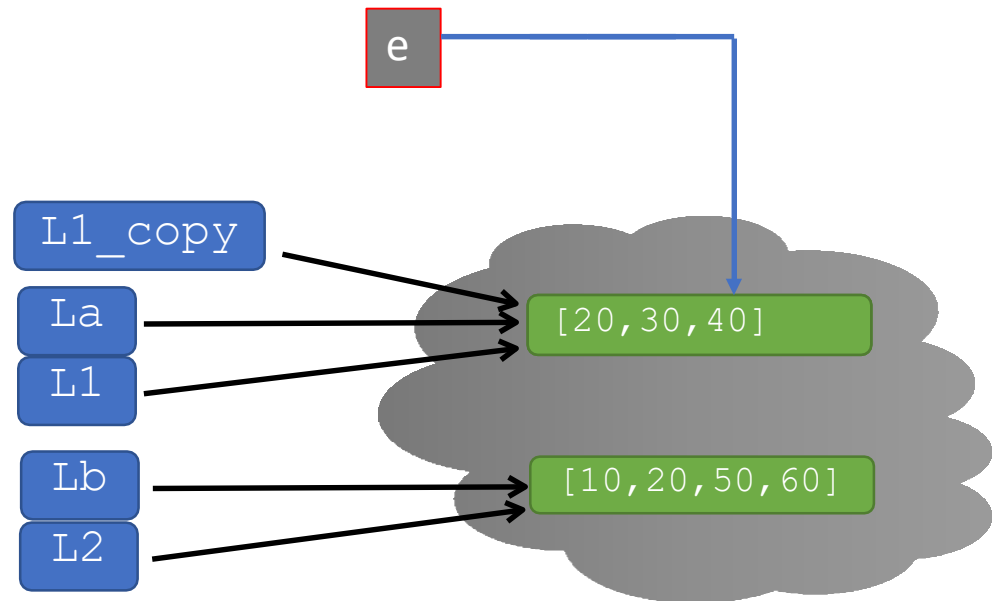
```
            L1.remove(e)
```

```
La = [10, 20, 30, 40]
```

```
Lb = [10, 20, 50, 60]
```

```
remove_dups(La, Lb)
```

```
print(La)
```



```
def remove_dups(L1, L2):
```

Alias for La

Alias for Lb

```
    L1_copy = L1
```

```
    for e in L1_copy:
```

```
        if e in L2:
```

```
            L1.remove(e)
```

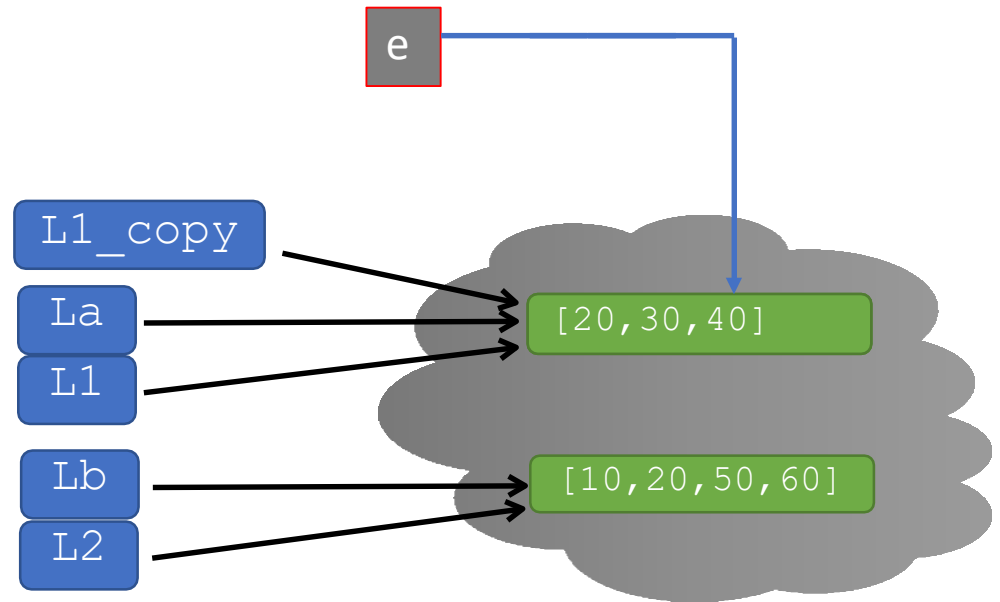
```
La = [10, 20, 30, 40]
```

```
Lb = [10, 20, 50, 60]
```

```
remove_dups(La, Lb)
```

```
print(La)
```

*L1 was mutated, but
it's an alias for La*



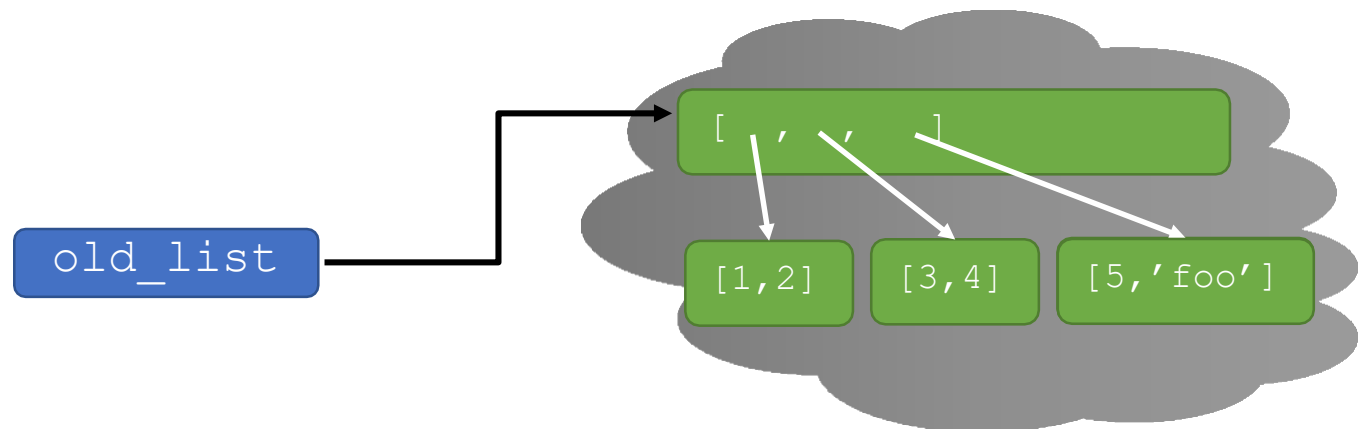
SHALLOW COPIES, DEEP COPIES WITH MUTABLE ELEMENTS

CONTROL COPYING

- ▶ Assignment just creates a new pointer to same object

```
old_list = [[1,2],[3,4],[5,'foo']]  
new_list = old_list
```

```
new_list[2][1] = 6  
print("New list:", new_list)  
print("Old list:", old_list)
```



CONTROL COPYING

- ▶ Assignment just creates a new pointer to same object

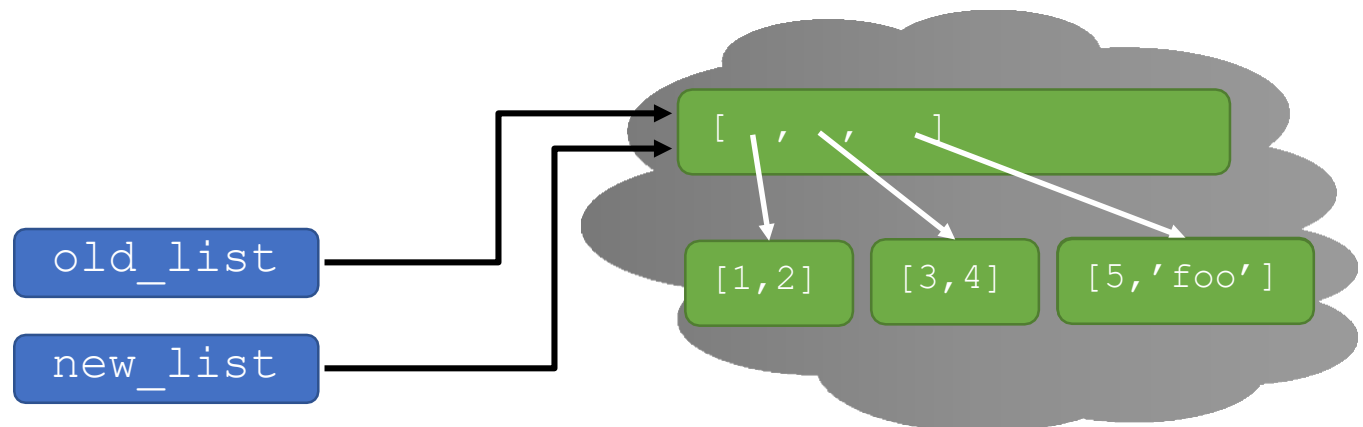
```
old_list = [[1,2],[3,4],[5,'foo']]
```

```
new_list = old_list
```

```
new_list[2][1] = 6
```

```
print("New list:", new_list)
```

```
print("Old list:", old_list)
```



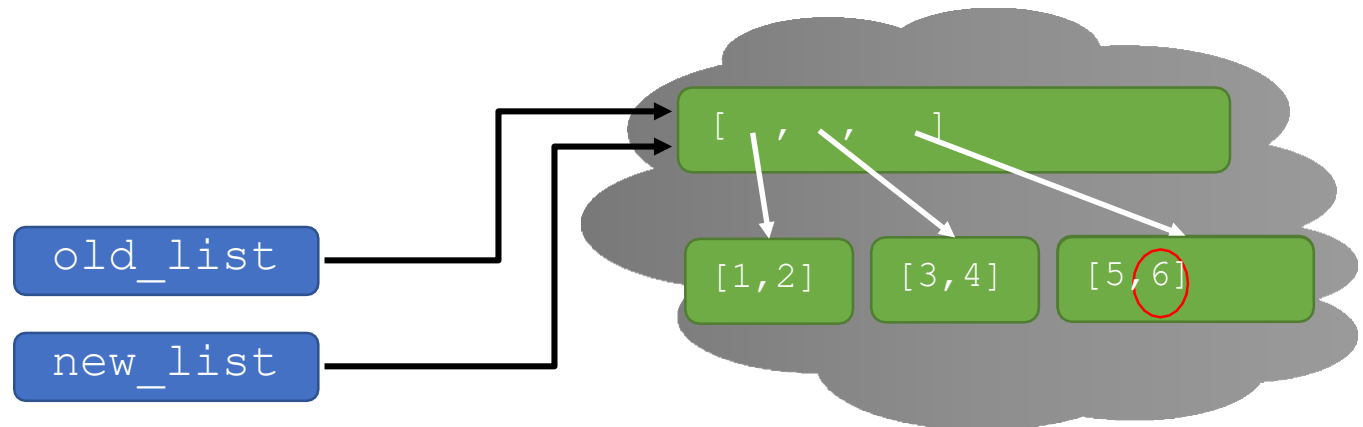
CONTROL COPYING

- ▶ Assignment just creates a new pointer to same object

```
old_list = [[1,2],[3,4],[5,'foo']]  
new_list = old_list
```

```
new_list[2][1] = 6
```

```
print("New list:", new_list)  
print("Old list:", old_list)
```



CONTROL COPYING

- ▶ Assignment just creates a new pointer to same object

```
old_list = [[1,2],[3,4],[5,'foo']]  
new_list = old_list
```

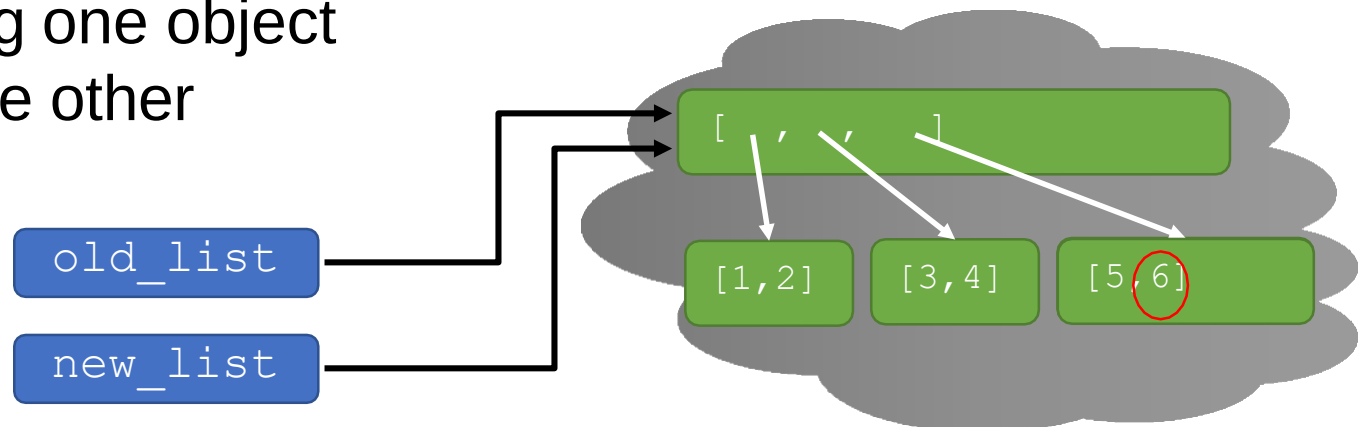
Assignment creates an alias for an existing data structure

```
new_list[2][1] = 6
```

```
print("New list:", new_list) # New list: [[1,2],[3,4],[5,6]]
```

```
print("Old list:", old_list) # Old list: [[1,2],[3,4],[5,6]]
```

- ▶ So mutating one object changes the other



CONTROL COPYING

- ▶ Create a copy of a list, not just a shared pointer:

```
import copy
```

```
L = [1,2,3,4]
```

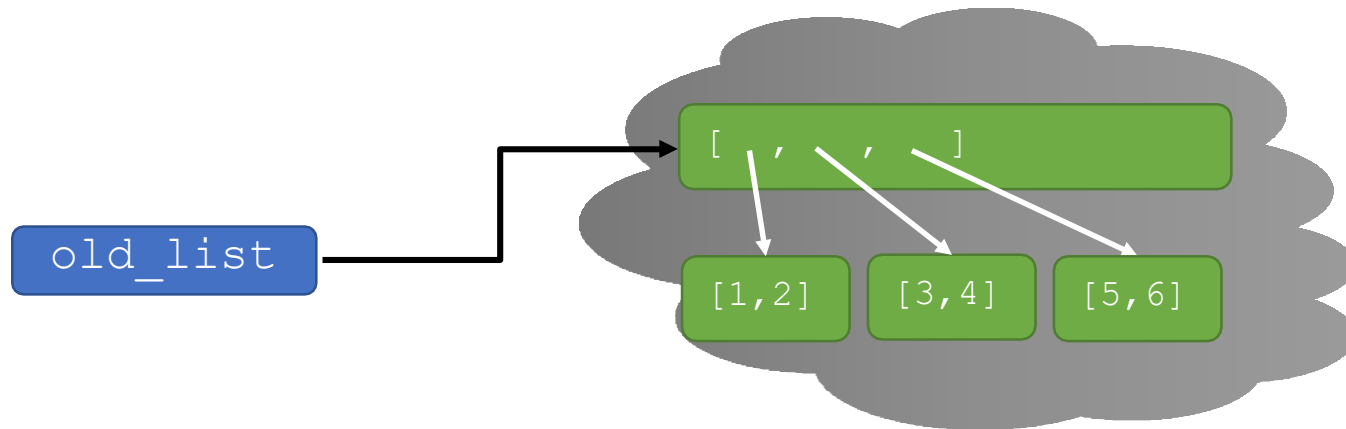
```
Lcopy = copy.copy(L) #equivalent to Lcopy = L[:]
```

- ▶ This is **shallow copying**
 - ▶ Copy only at the **top level of the list**
 - ▶ Elements are NOT copied
 - ▶ Use this when your list contains immutable objects only



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

```
print("New list:", new_list)  
print("Old list:", old_list)
```

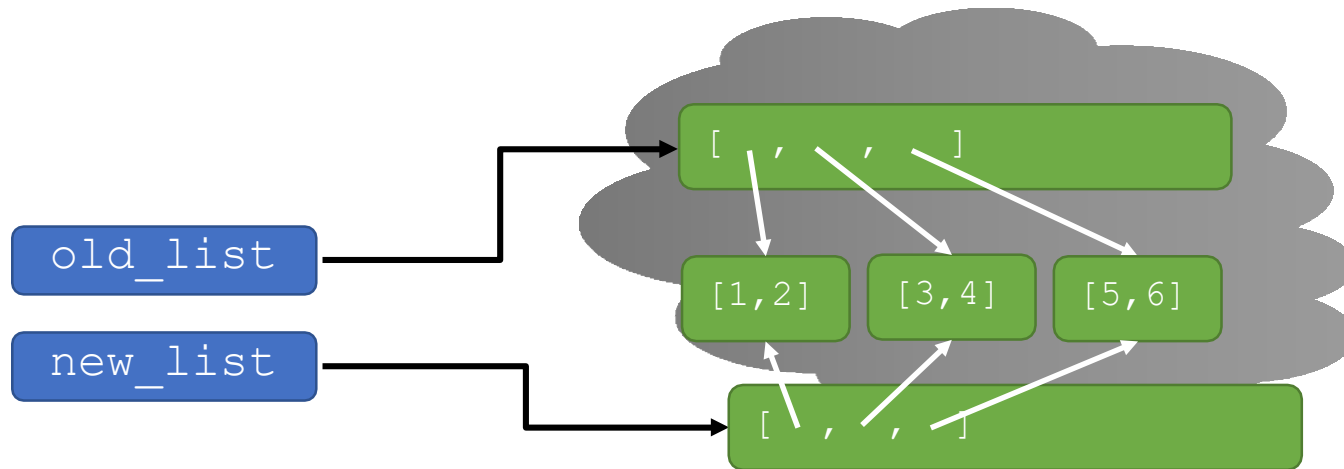


```
old_list = [[1,2],[3,4],[5,6]]
```

```
new_list = copy.copy(old_list)
```

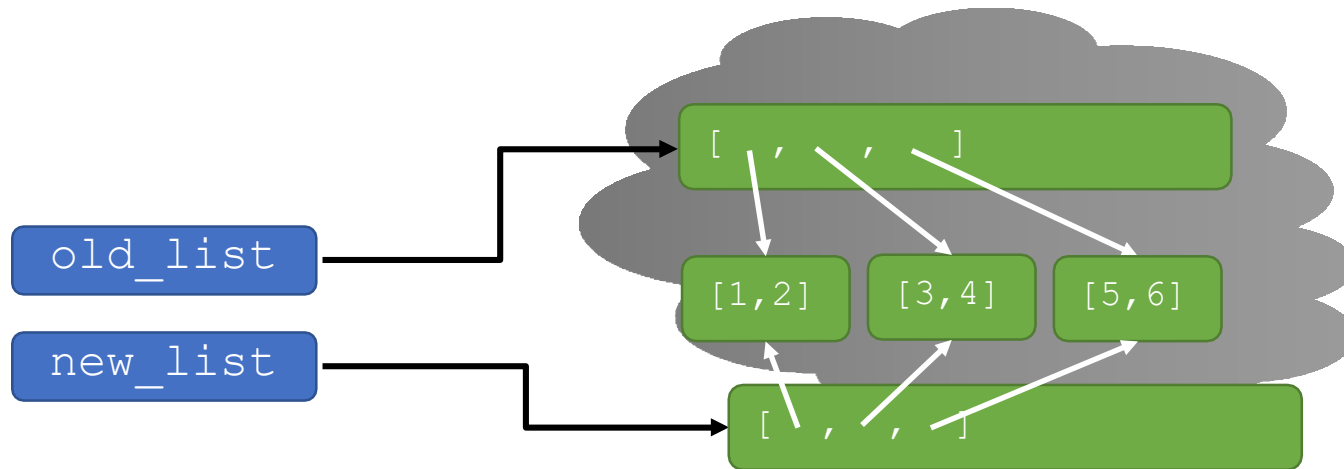
```
print("New list:", new_list)
```

```
print("Old list:", old_list)
```



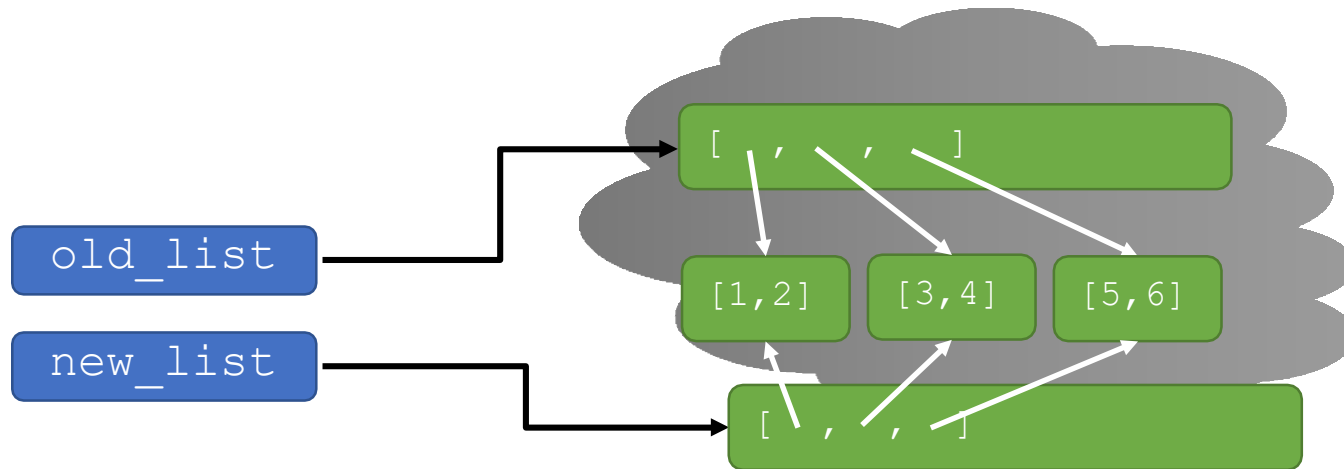
```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

```
print("New list:", new_list) New list: [[1,2],[3,4],[5,6]]  
print("Old list:", old_list)
```



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

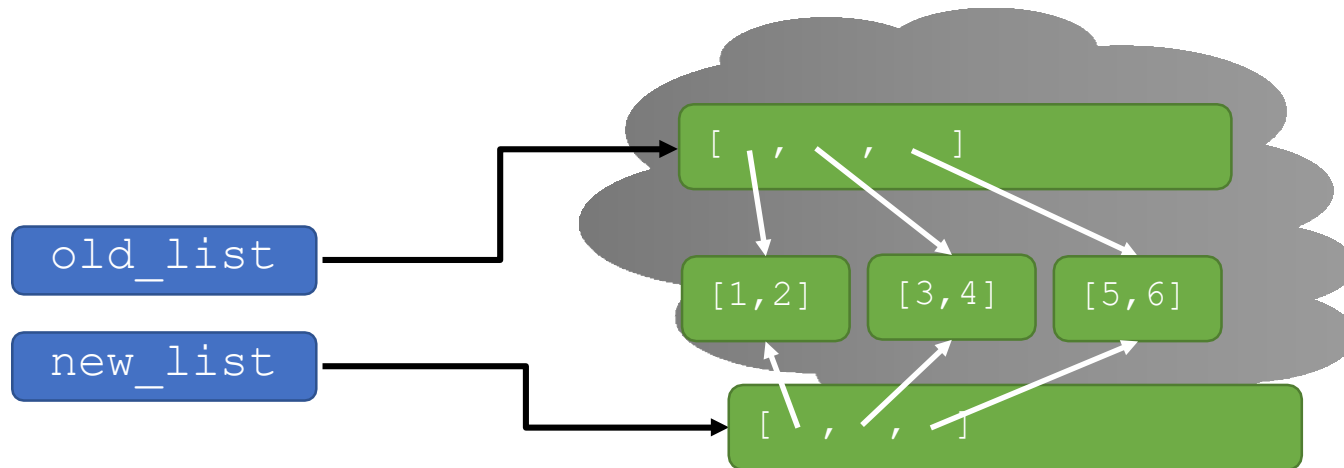
```
print("New list:", new_list) New list: [[1,2],[3,4],[5,6]]  
print("Old list:", old_list) Old list: [[1,2],[3,4],[5,6]]
```



```
old_list = [[1,2],[3,4],[5,6]]
new_list = copy.copy(old_list)
```

Copy creates a new data structure, but actual elements are shared

```
print("New list:", new_list) New list: [[1,2],[3,4],[5,6]]
print("Old list:", old_list) Old list: [[1,2],[3,4],[5,6]]
```



CONTROL COPYING

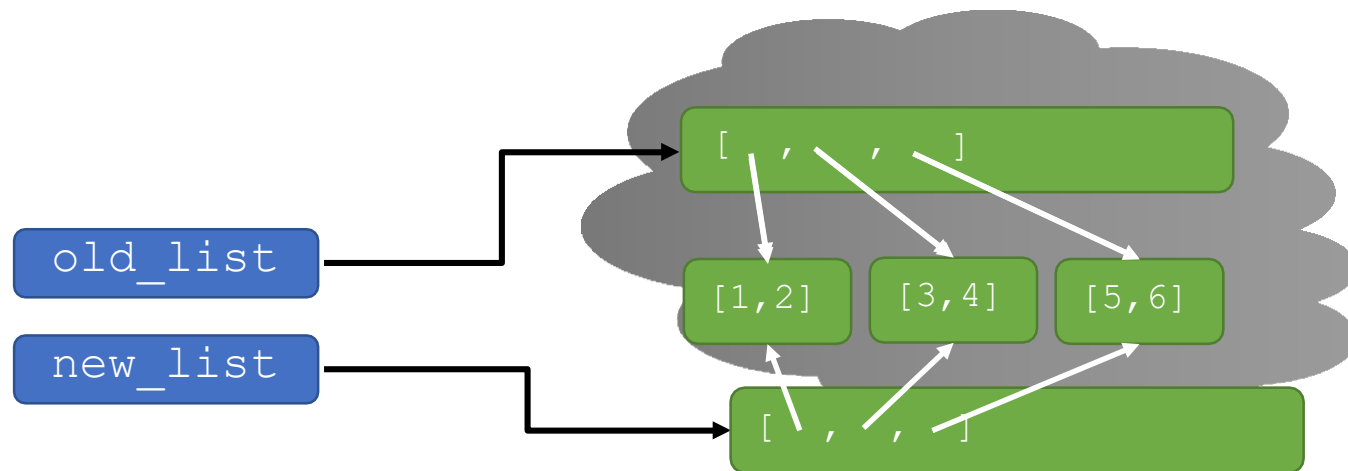
► Now we mutate the top level structure

```
import copy  
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)  
  
old_list.append([7,8])  
print("New list:", new_list)  
print("Old list:", old_list)
```



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

```
old_list.append([7,8])  
print("New list:", new_list)  
print("Old list:", old_list)
```

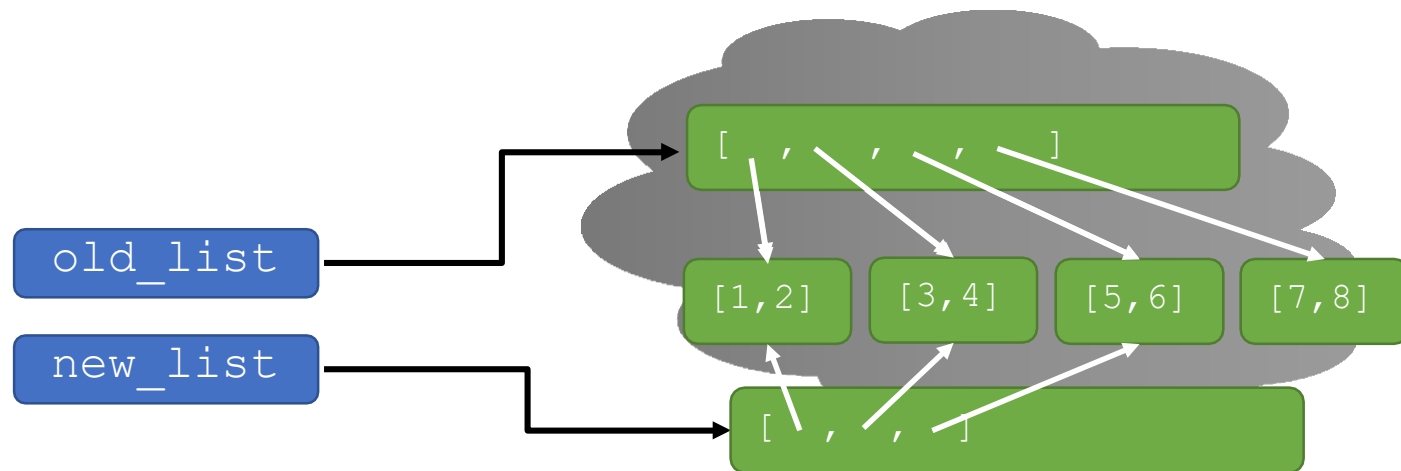


```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

```
old_list.append([7,8])
```

```
print("New list:", new_list)
```

```
print("Old list:", old_list)
```



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

*Copy creates a new data structure
with shared elements, so
mutating top level structure of
one does not affect the clone*

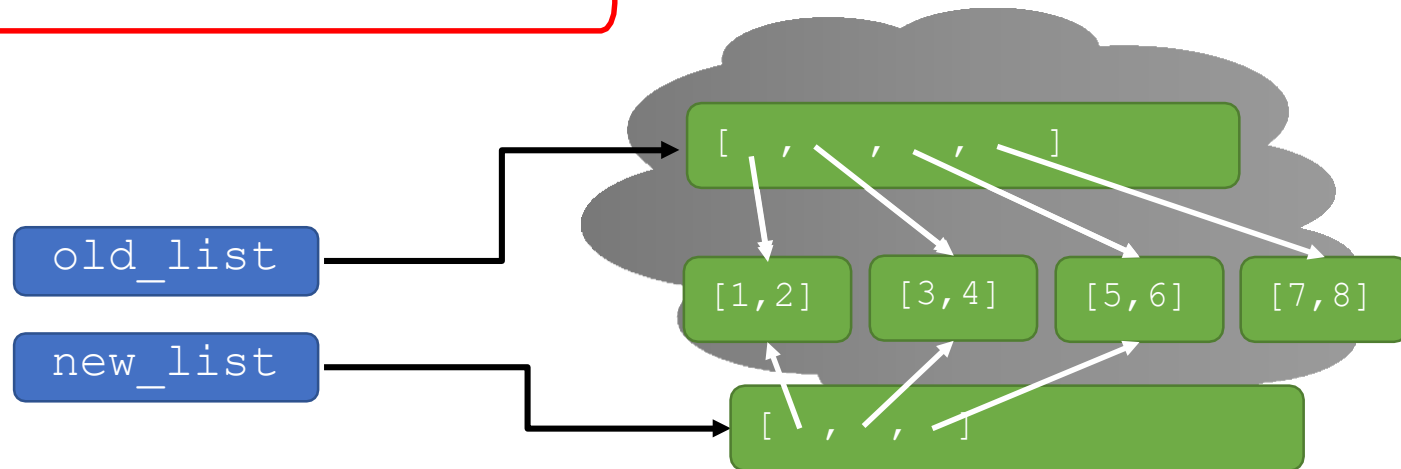
```
old_list.append([7,8])
```

```
print("New list:", new_list)
```

New list: [[1,2],[3,4],[5,6]]

```
print("Old list:", old_list)
```

Old list: [[1,2],[3,4],[5,6],[7,8]]



CONTROL COPYING

- ▶ But if we change an element in one of the sub-structures, they are shared!
 - ▶ Unless your elements are not mutable, then this is not a problem

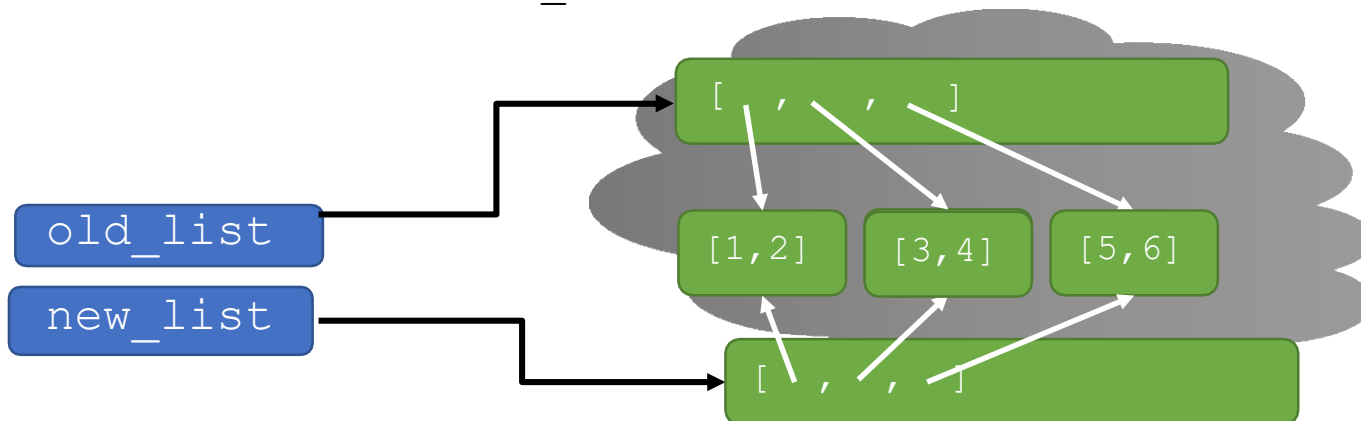
```
import copy
old_list = [[1,2],[3,4],[5,6]]
new_list = copy.copy(old_list)
```

```
old_list.append([7,8])
old_list[1][1] = 9
print("New list:", new_list)
print("Old list:", old_list)
```



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

```
old_list.append([7,8])  
old_list[1][1] = 9  
print("New list:", new_list)  
print("Old list:", old_list)
```



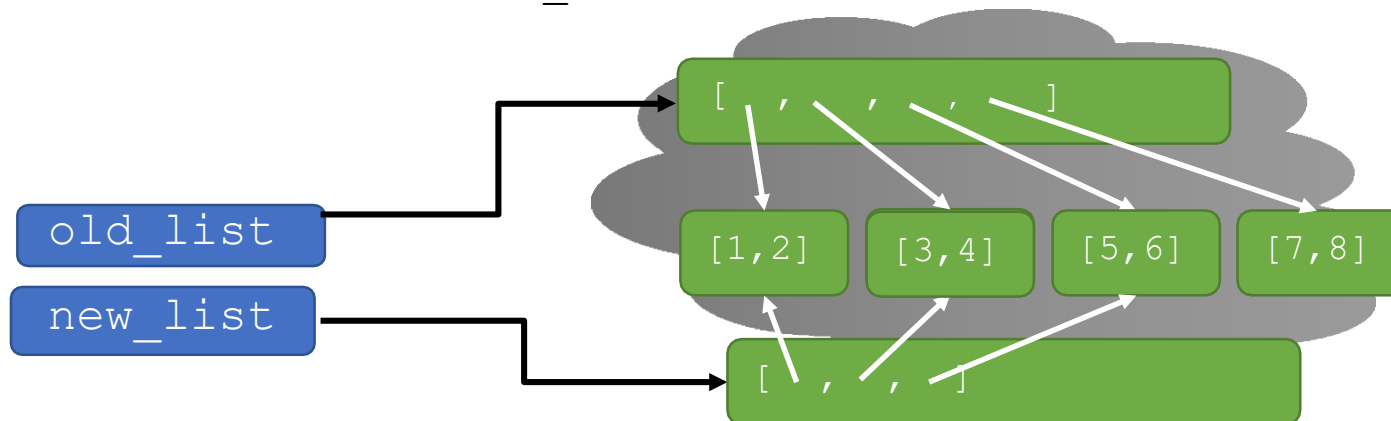
```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

```
old_list.append([7,8])
```

```
old_list[1][1] = 9
```

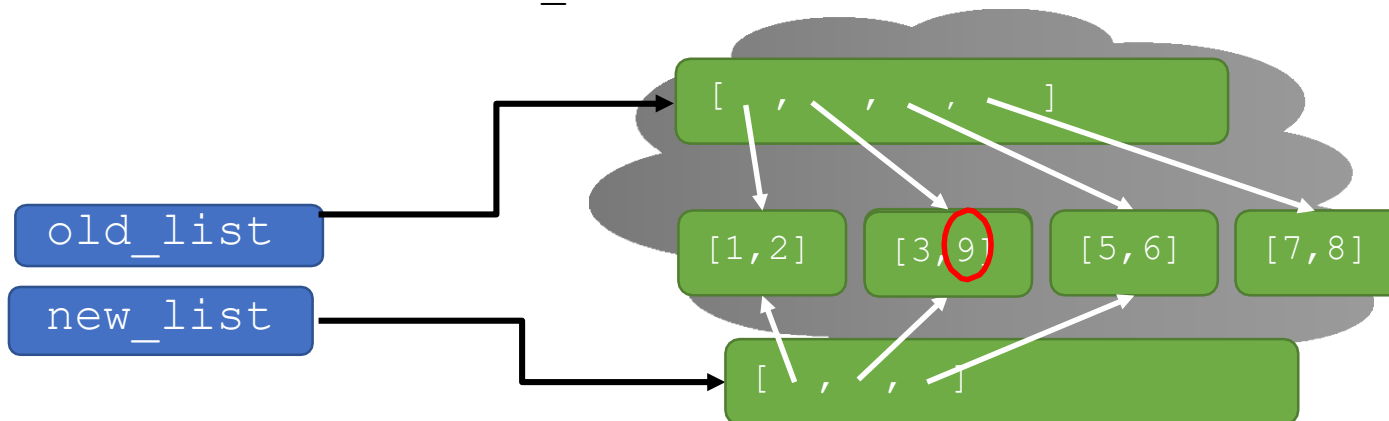
```
print("New list:", new_list)
```

```
print("Old list:", old_list)
```



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

```
old_list.append([7,8])  
old_list[1][1] = 9  
print("New list:", new_list)  
print("Old list:", old_list)
```



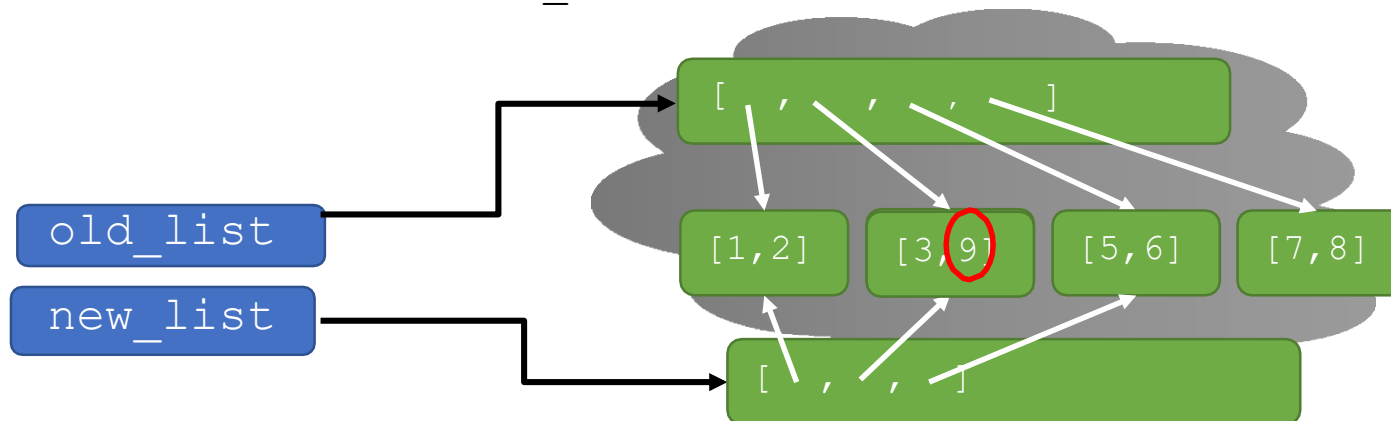
```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.copy(old_list)
```

```
old_list.append([7,8])
```

```
old_list[1][1] = 9
```

```
print("New list:", new_list) New list: [[1,2],[3,9],[5,6]]
```

```
print("Old list:", old_list)
```

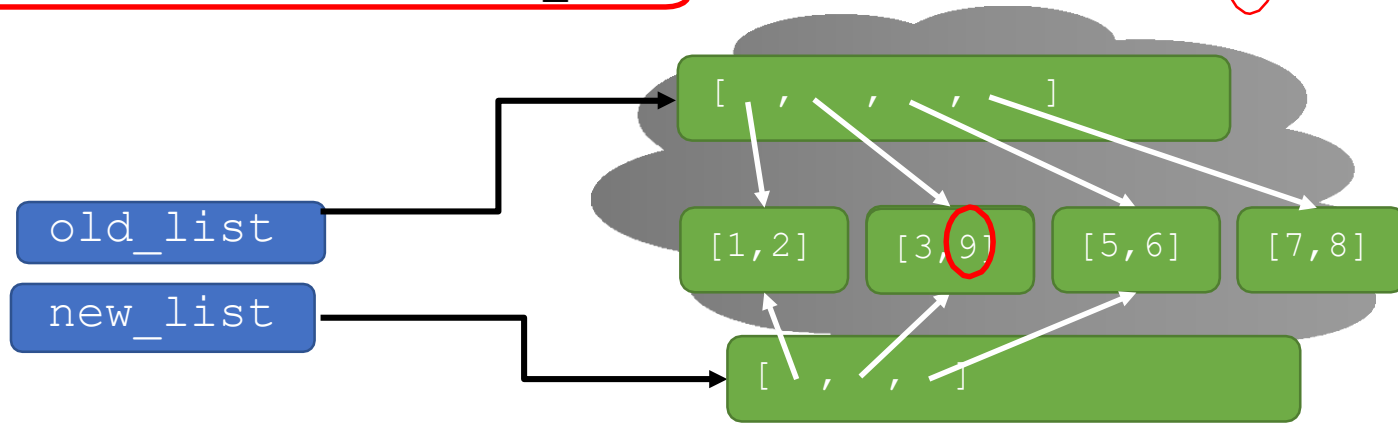


```
old_list = [[1,2],[3,4],[5,6]]
new_list = copy.copy(old_list)
```

Shallow copying creates a new data structure with shared elements; so top level are clones, but elements are aliases; mutating an element will thus affect the other object

```
old_list.append([7,8])
old_list[1][1] = 9
```

```
print("New list:", new_list) New list: [[1,2],[3,9],[5,6]]
print("Old list:", old_list) Old list: [[1,2],[3,9],[5,6],[7,8]]
```



CONTROL COPYING

- ▶ If we want all structures to be new copies, we need a **deep copy**
- ▶ Use deep copy when your list might have mutable elements to ensure every structure at every level is copied

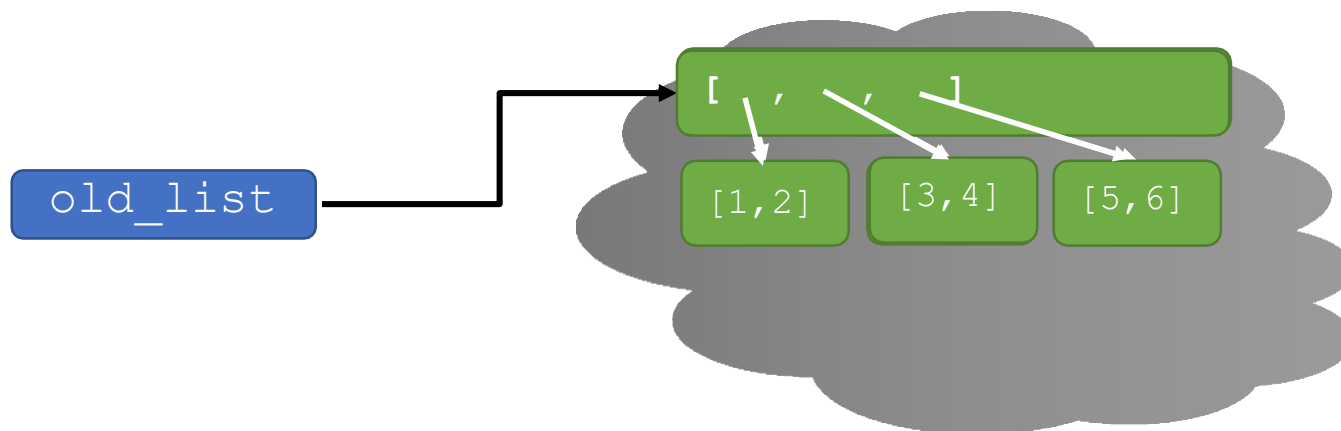
```
import copy  
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.deepcopy(old_list)
```

```
old_list.append([7,8])  
old_list[1][1] = 9  
  
print("New list:", new_list)  
print("Old list:", old_list)
```



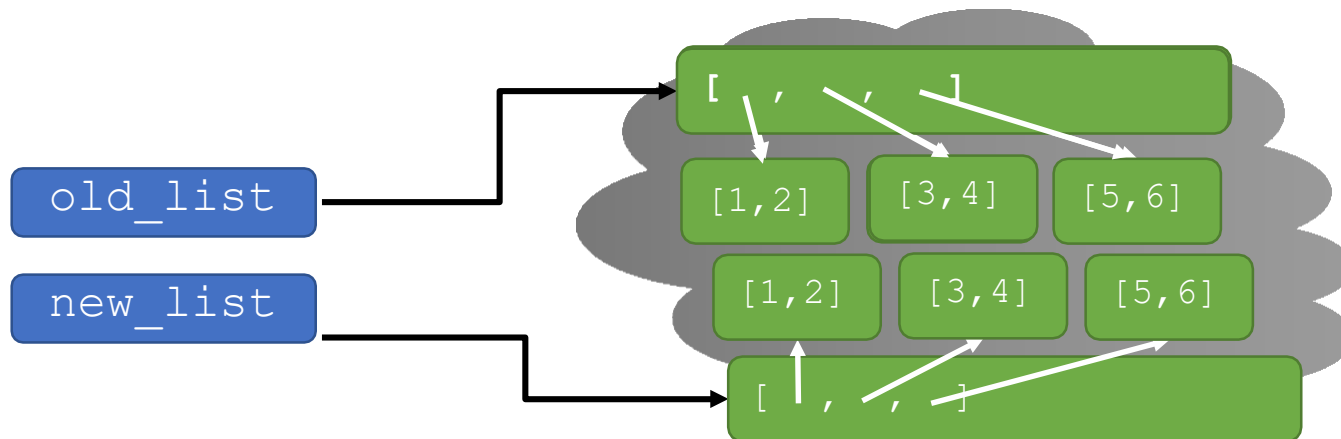
```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.deepcopy(old_list)
```

```
old_list.append([7,8])  
old_list[1][1] = 9  
print("New list:", new_list)  
print("Old list:", old_list)
```



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.deepcopy(old_list)
```

```
old_list.append([7,8])  
old_list[1][1] = 9  
print("New list:", new_list)  
print("Old list:", old_list)
```



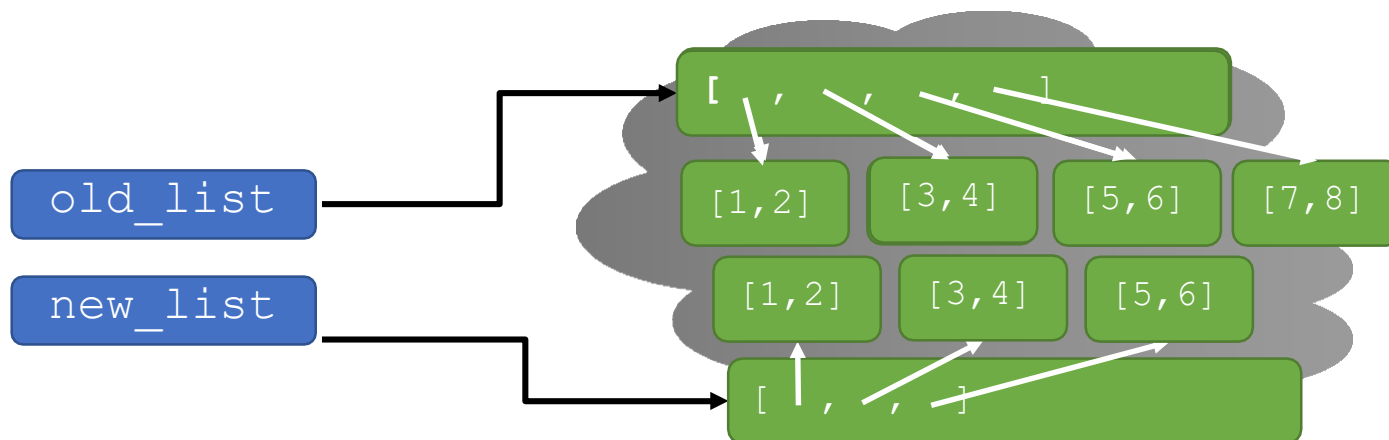
```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.deepcopy(old_list)
```

```
old_list.append([7,8])
```

```
old_list[1][1] = 9
```

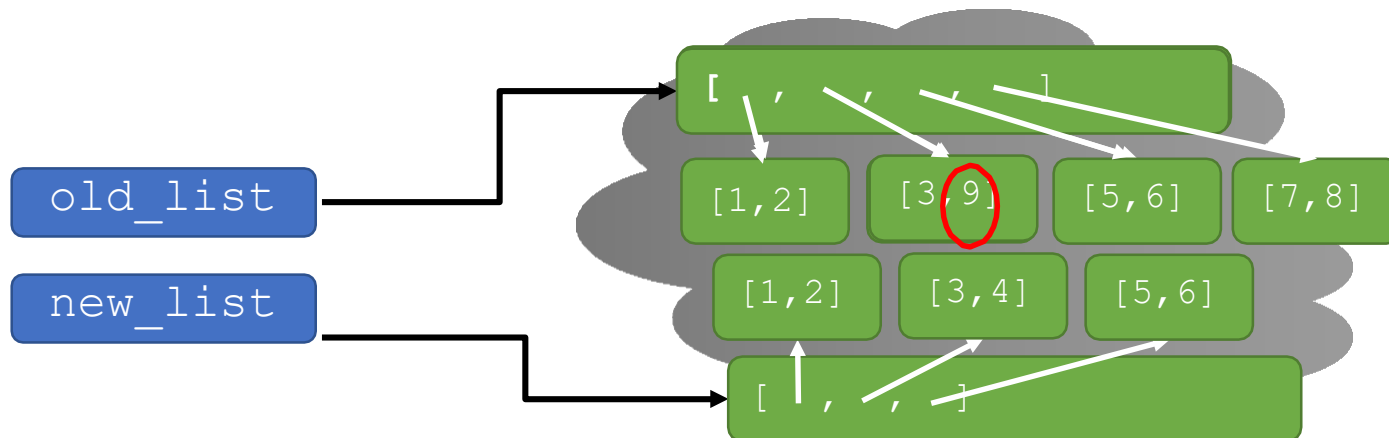
```
print("New list:", new_list)
```

```
print("Old list:", old_list)
```



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.deepcopy(old_list)
```

```
old_list.append([7,8])  
old_list[1][1] = 9  
print("New list:", new_list)  
print("Old list:", old_list)
```



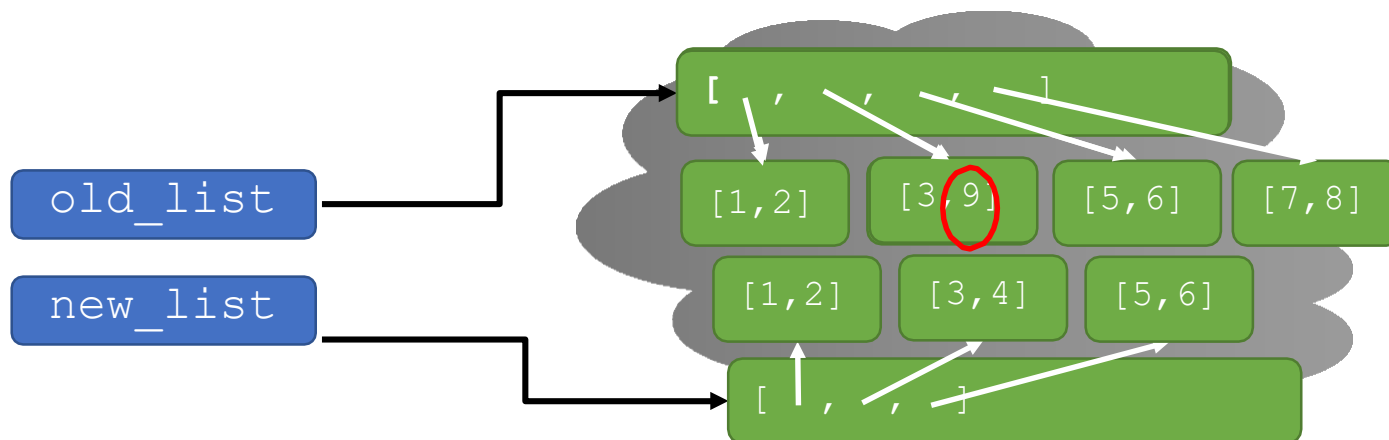
```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.deepcopy(old_list)
```

```
old_list.append([7,8])
```

```
old_list[1][1] = 9
```

```
print("New list:", new_list) New list: [[1,2],[3,4],[5,6]]
```

```
print("Old list:", old_list)
```



```
old_list = [[1,2],[3,4],[5,6]]  
new_list = copy.deepcopy(old_list)
```

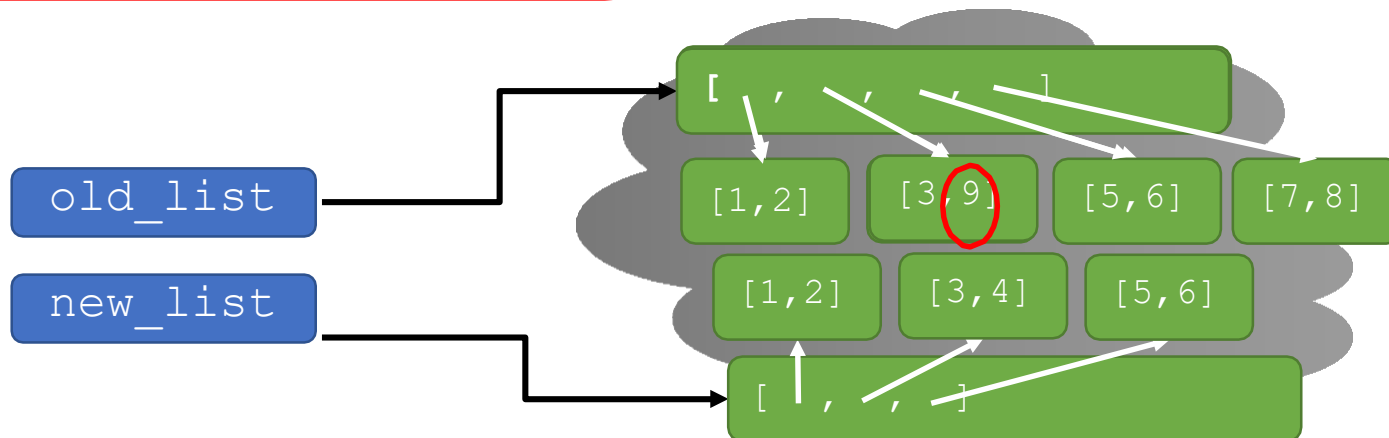
```
old_list.append([7,8])
```

```
old_list[1][1] = 9
```

```
print("New list:", new_list) New list: [[1,2],[3,4],[5,6]]
```

```
print("Old list:", old_list) Old list: [[1,2],[3,9],[5,6],[7,8]]
```

Deep copying creates clones at all levels of structure; thus mutating one does not affect the other at any level



Summary: LISTS in MEMORY

- ▶ Separate the idea of the **object vs. the name we give** an object
 - ▶ A list is an object in memory
 - ▶ Variable name points to object
- ▶ Lists are **mutable** and behave differently than immutable types
- ▶ Using **equal sign** between mutable objects creates **aliases**
 - ▶ Both variables point to the same object in memory
 - ▶ Any variable pointing to that object is affected by mutation of object, even if mutation is by referencing another name
- ▶ If you want a copy, you explicitly tell Python to make a copy
 - ▶ **Shallow vs. deep copy**



WHY LISTS and TUPLES?

- ▶ If mutation can cause so many problems, why do we even want to have lists, **why not just use tuples?**
 - ▶ Efficiency – if processing very large sequences, don't want to have to copy every time we change an element
- ▶ If lists basically do everything that tuples do, **why not just have lists?**
 - ▶ Immutable structures can be very valuable in context of other object types
 - ▶ Don't want to accidentally have other code mutate some important data; tuples safeguard against this
 - ▶ They can be a bit faster





LIST COMPREHENSION

LIST COMPREHENSION

- ▶ Applying a **function to every element of a sequence**, then creating a new list with these values is a common practice

- ▶ Example:

```
def f(L):  
    Lnew = []  
    for e in L:  
        Lnew.append(e**2)  
    return Lnew
```

New list

Function to apply

- ▶ Python provides a concise one-liner way to do this, called a **list comprehension**
 - ▶ Creates a new list
 - ▶ Applies a function to every element of another iterable
 - ▶ Optional, only apply to elements that satisfy a test

```
[expression for elem in iterable if test]
```



LIST COMPREHENSION

- ▶ Create a new list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):  
    Lnew = []  
    for e in L:  
        Lnew.append(e**2)  
    return Lnew
```

New list

New list

```
Lnew = [e**2 for e in L]
```



LIST COMPREHENSION

- ▶ Create a new list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):  
    Lnew = []  
    for e in L:  
        Lnew.append(e**2)  
    return Lnew
```

Function
to apply

```
Lnew = [e**2 for e in L]
```



LIST COMPREHENSION

- ▶ Create a new list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):  
    Lnew = []  
    for e in L:  
        Lnew.append(e**2)  
    return Lnew
```

Look at every element

```
Lnew = [e**2 for e in L]
```



LIST COMPREHENSION

- ▶ Create a new list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):  
    Lnew = []  
    for e in L:  
        Lnew.append(e**2)  
    return Lnew
```

```
def f(L):  
    Lnew = []  
    for e in L:  
        if e%2==0:  
            Lnew.append(e**2)  
    return Lnew
```

`Lnew = [e**2 for e in L]`

New list

`Lnew = [e**2 for e in L if e%2==0]`



LIST COMPREHENSION

- ▶ Create a new list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):  
    Lnew = []  
    for e in L:  
        Lnew.append(e**2)  
    return Lnew
```

```
def f(L):  
    Lnew = []  
    for e in L:  
        if e%2==0:  
            Lnew.append(e**2)  
    return Lnew
```

*Look over
elements*

```
Lnew = [e**2 for e in L]
```

```
Lnew = [e**2 for e in L if e%2==0]
```



LIST COMPREHENSION

- ▶ Create a new list, by applying a function to every element of another iterable that satisfies a test

```
def f(L):  
    Lnew = []  
    for e in L:  
        Lnew.append(e**2)  
    return Lnew
```

```
def f(L):  
    Lnew = []  
    for e in L:  
        if e%2==0:  
            Lnew.append(e**2)  
    return Lnew
```

*Function to apply
only if test is true*

```
Lnew = [e**2 for e in L]
```

```
Lnew = [e**2 for e in L if e%2==0]
```



LIST COMPREHENSION

`[expression for elem in iterable if test]`

- ▶ This is equivalent to invoking this function (where expression is a function that computes that expression)

```
def f(expr, old_list, test = lambda x: True):  
    new_list = []  
    for e in old_list:  
        if test(e):  
            new_list.append(expr(e))  
    return new_list
```

`[e**2 for e in range(6)]` → `[0, 1, 4, 9, 16, 25]`

`[e**2 for e in range(8) if e%2 == 0]` → `[0, 4, 16, 36]`

`[[e,e**2] for e in range(4) if e%2 != 0]` → `[[1,1], [3,9]]`



YOU TRY IT!

- ▶ What is the value returned by this expression?
 - ▶ Step1: what are **all values** in the sequence
 - ▶ Step2: which **subset of values** does the condition filter out?
 - ▶ Step3: **apply the function** to those values

```
[len(x) for x in ['xy', 'abcd', 7, '4.0'] if type(x) == str]
```





TESTING and DEBUGGING

TESTING and DEBUGGING

- ▶ Three steps of programming
 - ▶ Coding
 - ▶ Testing/Validation
 - ▶ Debugging, if there're errors



SET YOURSELF UP FOR EASY TESTING AND DEBUGGING

- ▶ From the **start**, design code to ease this part
- ▶ Break program up into **modules** that can be tested and debugged individually
- ▶ **Document** what modules do
- ▶ **Document constraints** on modules
 - ▶ What do you expect the input/output to be?
- ▶ **Document assumptions** behind code design



WHEN ARE YOU READY TO TEST?

- ▶ Ensure **code runs**
 - ▶ Remove syntax errors
 - ▶ Remove static semantic errors
 - ▶ Python interpreter can usually find these for you
- ▶ Have a **set of expected results** (test cases)
 - ▶ An input set
 - ▶ For each input, the expected output
 - ▶ Shall include boundary cases (empty lists, singleton list, large numbers, small numbers)



CLASSES OF TESTS

▶ **Unit testing**

- ▶ Validate each piece of program
- ▶ Testing each function separately

▶ **Regression testing**

- ▶ Re-run tests after a change
- ▶ Catch reintroduced errors that were previously fixed

▶ **Integration testing**

- ▶ Does **overall program** work?
- ▶ Tend to rush to do this



DEBUGGING

- ▶ Once you have discovered that your code does not run properly, you want to:
 - ▶ Isolate the bug(s)
 - ▶ Eradicate the bug(s)
 - ▶ Retest until code runs correctly for all cases
- ▶ Tools
 - ▶ **Built in** to IDE
 - ▶ **print** statement
 - ▶ Use your brain, be **systematic** in your hunt
- ▶ Use **bisection method**
 - ▶ Put print or breakpoint halfway in code
 - ▶ ...so you know which part is buggy





EXCEPTIONS

UNEXPECTED CONDITIONS

- ▶ What happens when procedure execution hits an **unexpected condition**?
- ▶ Get an **exception**... to what was expected

- ▶ Trying to access beyond list limits

```
test = [1,7,4]
```

```
test[4]
```

→ `IndexError`

- ▶ Trying to convert an inappropriate type

```
int(test)
```

→ `TypeError`

- ▶ Referencing a non-existing variable

```
a
```

→ `NameError`

- ▶ Mixing data types without coercion

```
'a' / 4
```

→ `TypeError`



HANDLING EXCEPTIONS

- ▶ Typically, exception causes an error to occur and execution to stop
- ▶ Python code can provide **handlers** for exceptions

```
try:
```

```
    # do some potentially  
    # problematic code
```

```
except:
```

```
    # do something to  
    # handle the problem
```

- ▶ If expressions in **try block all succeed**
 - ▶ Evaluation continues with code after except block
- ▶ Exceptions **raised** by any statement in body of **try** are **handled** by the **except** statement
 - ▶ Execution continues with the body of the except statement
 - ▶ Then other expressions after that block of code



EXAMPLE

- ▶ A function that sums digits in a string

Code without exceptions

```
def sum_digits(s):  
    """ s is a non-empty string  
        containing digits.  
    Returns sum of all chars that  
    are digits """  
    total = 0  
    for char in s:  
        if char in '0123456789':  
            val = int(char)  
            total += val  
    return total
```

*Problematic if try to do
int('a')*

Code with exceptions

```
def sum_digits(s):  
    """ s is a non-empty string  
        containing digits.  
    Returns sum of all chars that  
    are digits """  
    total = 0  
    for char in s:  
        try:  
            val = int(char)  
            total += val  
        except:  
            print("can't convert", char)  
    return total
```

*Print and move
on to next char*



EXAMPLE

```
a = int(input("Tell me one number:"))  
b = int(input("Tell me another number:"))  
print(a/b)
```

- ▶ User might input a character :(
- ▶ User might make b be 0 :(
- ▶ Use try/except around the problematic code

try:

```
a = int(input("Tell me one number:"))  
b = int(input("Tell me another number:"))  
print(a/b)
```

except:

```
print("Bug in user input.")
```



HANDLING SPECIFIC EXCEPTIONS

- ▶ Have **separate except clauses** to deal with a particular type of exception

```
try:
    a = int(input("Tell me one number: "))
    b = int(input("Tell me another number: "))
    print("a/b = ", a/b)
    print("a+b = ", a+b)
except ValueError:
    print("Could not convert to a number.")
except ZeroDivisionError:
    print("Can't divide by zero")
    print("a/b = infinity")
    print("a+b =", a+b)
except:
    print("Something went very wrong.")
```

*only execute
if these errors
come up*

*for all other
errors*



OTHER BLOCKS ASSOCIATED WITH A TRY BLOCK

- ▶ `else:`

- ▶ Body of this is executed when execution of associated `try` body **completes with no exceptions**

- ▶ `finally:`

- ▶ Body of this is **always executed** after `try`, `else` and `except` clauses, even if they raised another error or executed a `break`, `continue` or `return`

- ▶ Useful for clean-up code that should be run no matter what else happened (e.g. close a file)

- ▶ Nice to know these exist, but we don't really use these in this class



WHAT TO DO WITH EXCEPTIONS?

- ▶ What to do when encounter an error?
- ▶ **Fail silently:**
 - ▶ Substitute default values or just continue
 - ▶ Bad idea! user gets no warning
- ▶ Return an **“error” value**
 - ▶ What value to choose?
 - ▶ Complicates code having to check for a special value
- ▶ Stop execution, **signal error** condition
 - ▶ In Python: **raise an exception**

```
raise ValueError("something is wrong")
```

keyword

name of error
you want to raise

optional, but typically a
string with a message



EXAMPLE with SOMETHING YOU'VE ALREADY SEEN

- ▶ A function that sums digits in a string
- ▶ Execution stopping means a bad result is not propagated

```
def sum_digits(s):  
    """ s is a non-empty string containing digits.  
    Returns sum of all chars that are digits """  
    assert len(s) != 0, "s is empty"  
    total = 0  
    for char in s:  
        try:  
            val = int(char)  
            total += val  
        except:  
            raise ValueError("string contained a character")
```



YOU TRY IT!

```
def pairwise_div(Lnum, Ldenom):  
    """ Lnum and Ldenom are non-empty lists of equal lengths  
    containing numbers  
  
    Returns a new list whose elements are the pairwise  
    division of an element in Lnum by an element in Ldenom.  
    Raise a ValueError if Ldenom contains 0. """  
    # your code here  
  
# For example:  
L1 = [4,5,6]  
L2 = [1,2,3]  
# print(pairwise_div(L1, L2)) # prints [4.0,2.5,2.0]  
  
L1 = [4,5,6]  
L2 = [1,0,3]  
# print(pairwise_div(L1, L2)) # raises a ValueError
```





ASSERTIONS

ASSERTIONS: DEFENSIVE PROGRAMMING TOOL

- ▶ Want to be sure that **assumptions** on state of computation are as expected
- ▶ Use an **assert statement** to raise an AssertionError exception if assumptions not met

```
assert <statement that should be true>, "message  
if not true"
```

- ▶ Assertions don't allow a programmer to control response to unexpected conditions
- ▶ Ensure that **execution halts** whenever an expected condition is not met
- ▶ Often used to **check inputs** to functions
- ▶ Can be used to **check outputs** of a function to avoid propagating bad values
- ▶ Can make it easier to locate a source of a bug
- ▶ An example of good **defensive programming**



EXAMPLE with SOMETHING YOU'VE ALREADY SEEN

- ▶ A function that sums digits in a **NON-EMPTY string**
- ▶ Execution stopping means a bad result is not propagated

```
def sum_digits(s):  
    """ s is a non-empty string containing digits.  
    Returns sum of all chars that are digits """  
    assert len(s) != 0, "s is empty"  
    total = 0  
    for char in s:  
        try:  
            val = int(char)  
            total += val  
        except:  
            raise ValueError("string contained a character")
```



YOU TRY IT!

```
def pairwise_div(Lnum, Ldenom):  
    """ Lnum and Ldenom are non-empty lists of equal lengths.  
    Returns a new list whose elements are the pairwise  
    division of an element in Lnum by an element in Ldenom.  
    Raise a ValueError if Ldenom contains 0. """  
    # add an assert line here
```



LONGER EXAMPLE OF EXCEPTIONS and ASSERTIONS

- ▶ Assume we are given **a class list**: each entry is a list of two parts
 - ▶ A list of first and last name for a student
 - ▶ A list of grades on assignments

Two students, each with a name list and a grades list

```
test_grades = [[['peter', 'parker'], [80.0, 70.0, 85.0]],  
               [['bruce', 'wayne'], [100.0, 80.0, 74.0]]]
```

- ▶ Create **a new class list**, with name, grades, and an average added at the end

```
[[['peter', 'parker'], [80.0, 70.0, 85.0], 78.33333],  
 [['bruce', 'wayne'], [100.0, 80.0, 74.0], 84.666667]]]
```



EXAMPLE CODE

```
[[['peter', 'parker'], [80.0, 70.0, 85.0]],  
 [['bruce', 'wayne'], [100.0, 80.0, 74.0]]]
```

```
def get_stats(class_list):  
    new_stats = []  
    for stu in class_list:  
        new_stats.append([stu[0], stu[1], avg(stu[1])])  
    return new_stats
```

```
def avg(grades):  
    return sum(grades)/len(grades)
```

*We will look at variations
of the avg function*



ERROR IF NO GRADE FOR A STUDENT

- ▶ If one or more students **don't have any grades**, get an error

```
test_grades = [[['peter', 'parker'], [10.0, 55.0, 85.0]],  
               [['bruce', 'wayne'], [10.0, 80.0, 75.0]],  
               [['captain', 'america'], [80.0, 10.0, 96.0]],  
               [['deadpool'], []]]
```

- ▶ Get `ZeroDivisionError`: float division by zero because try to

```
return sum(grades) / len(grades)
```

length is 0



OPTION 1: FLAG THE ERROR BY PRINTING A MESSAGE

- ▶ Decide to **notify** that something went wrong with a msg

```
def avg(grades):  
    try:  
        return sum(grades)/len(grades)  
    except ZeroDivisionError:  
        print('warning: no grades data')
```

- ▶ Running on same test data gives

```
warning: no grades data
```

```
[[['peter', 'parker'], [10.0, 55.0, 85.0], 50.0],  
 [['bruce', 'wayne'], [10.0, 80.0, 75.0], 55.0],  
 [['captain', 'america'], [80.0, 10.0, 96.0], 62.0],  
 [['deadpool'], [], None]]
```

flagged the error

*because avg did
not return anything
in the except*



OPTION 2: CHANGE THE POLICY

- ▶ Decide that a student with no grades gets a **zero**

```
def avg(grades):  
    try:  
        return sum(grades)/len(grades)  
    except ZeroDivisionError:  
        print('warning: no grades data')  
        return 0.0
```

- ▶ Running on same test data gives

```
warning: no grades data
```

```
[[['peter', 'parker'], [10.0, 55.0, 85.0], 50.0],  
 [['bruce', 'wayne'], [10.0, 80.0, 75.0], 55.0],  
 [['captain', 'america'], [80.0, 10.0, 96.0], 62]  
 [['deadpool'], [], 0.0]]
```

still flag the error

now avg returns 0



OPTION 3: HALT EXECUTION IF ASSERT IS NOT MET

- ▶ Raises an `AssertionError` if it is given an empty list for grades, prints out string message; stops execution
- ▶ Otherwise runs as normal

*function ends
immediately if
assertion not met*

```
def avg(grades):
```

```
    assert len(grades) != 0, 'no grades data'
```

```
    return sum(grades)/len(grades)
```



ASSERTIONS vs. EXCEPTIONS

- ▶ Goal is to **spot problems as soon as introduced** and make clear where they happened
- ▶ Exceptions provide a way of **handling unexpected input**
 - ▶ Use when you don't need to halt program execution
 - ▶ Ex: if users supplies bad data input
- ▶ Use **assertions**:
 - ▶ Enforce conditions on a “contract”
 - ▶ As a **supplement** to testing

